KANERVA: Token-Budgeted, Validity-Aware Memory for Coding Agents, Without an LLM in the Loop

Artem Katolikov*

October 2026

Abstract

Coding agents forget everything between sessions, and the memory systems built for them mostly select context by similarity top-k, ignore what items cost in tokens, and have no model of whether a stored fact is still true. We cast agent memory as online budgeted selection, show that the surveyed systems are special cases, and derive where each is worst-case suboptimal. We then present KANERVA, a local memory layer with no LLM in the loop. It scores each candidate with a calibrated relevance model over retrieval, item and context features, selects under a hard token budget, keeps an item's *validity* apart from its *usefulness*, expands candidates through neighbouring turns, and stays silent when the store looks unable to answer a prompt. We prove a $\frac{1}{2}(1-\frac{1}{e})$ selection guarantee with a per-query certificate, a bound on how probability error reaches the selection, a dynamic regret bound that separates the cost of expiring items, a linear lower bound for validity-blind learners, a rate–distortion bound for reversible consolidation, and a stationary store-size result. On a third held-out test with no LLM in the evaluation, on fifteen repositories in four languages that nothing was fitted or tuned on, averaged over budgets from 128 to 4096 tokens and without structured keys, KANERVA recovers 0.775 of the evidence against 0.752 for hybrid retrieval and 0.749 for hybrid with a cross-encoder, and it leads on all fifteen. On facts that changed it injects an outdated version 34.9% of the time against 99.9% for hybrid, and on none of the test queries with keys, though a simple keep-the-newest-twin rule gets 36.5% and the store alone does not detect free-text updates, though a real agent shown similar notes fixed most of them in a small trial. The lexical configuration, which needs no model, is close to level with BM25 from 512 tokens up. Three further tests on six C and C++ repositories each, which the model had never seen, are less clear. Two give the same picture with a smaller lead, a change to the near-twin rule cuts stale injection there to 16.0%, and on the third, with class methods among the facts, the lead over hybrid does not hold on strict recall. On BEAM, a check and not a test because earlier tests saw it, it recovers 0.574 against 0.440 for hybrid. No untouched conversation benchmark remains. A gate that keeps the store silent on prompts it cannot answer costs 5.6% of recall on the store's own questions, and its false-alarm rates come from foreign questions. When a small local model answers from the selected notes, little of this carries over: no difference between methods survives correction, and on conversations far higher recall does not beat BM25. Online learning adds almost nothing on top of the fitted model, on simulated feedback, and forgetting costs recall.

1 Introduction

A coding agent starts each session knowing nothing it learned in the last one. Native memory is a set of files loaded whole: Claude Code loads its `CLAUDE.md` files and the first 200 lines or 25 KB of a memory index, with no ranking by the task at hand [4]. Memory products add retrieval, but they select by similarity top-k or leave the choice to the agent, and they spend LLM calls on every write. The published evidence does not show that this is good enough. On LoCoMo, Mem0's own table has a full-context baseline ahead of every memory system tested [13]. Retrieval method moves accuracy far more than write strategy, and raw chunks with no LLM calls match fact extraction [70]. On coding tasks, unfiltered experience gives little or negative benefit [76], and eleven of twelve pairings of solvers with memory systems fail to beat memory-off [17].

These findings point at the read path, and at three defects in it. Selection ignores what an item costs in tokens. It ignores what the other selected items already say. And it ignores whether an item is still true, which is the common failure in a codebase, where a function signature read last month may have changed since.

This paper treats memory as an online budgeted selection problem and makes four contributions.

1. A framework in which the surveyed systems are special cases, with elementary worst-case arguments for what each one gives up (Section 3).

2. A method that scores each candidate with a cali-

*Code and data pointers: `https://github.com/katolikov/kanerva`. DOI: 10.5281/zenodo.23849911. ORCID: 0009-0007-1585-8092.

brated relevance model and selects under a hard token budget, with validity modelled apart from usefulness and estimated from the store's own version history (Section 4). It runs locally and needs no LLM.

3. A regret decomposition for items that expire (Theorem 2). Stationary linear submodular bandit results exist [58, 69, 71]; they exclude this case. Here non-stationarity costs the accumulated validity error, while a validity-blind learner has linear regret (Proposition 9). Proposition 7 bounds how probability error reaches the selection, and the other theory results are a rate–distortion bound for reversible consolidation and a stability result for store size.

4. RepoMem, a coding-memory benchmark built from the real histories of open-source repositories, in which facts change for real and every label comes from the code by static analysis, or from commit history outside Python. We evaluate on three held-out sets of RepoMem repositories, the last in four languages, and on BEAM conversations, against a dozen reading baselines, with ablations, confidence intervals and the cases where the method loses.

Scope of the evidence. Nearly all results here are *retrieval-level*: evidence recall under a token budget, tokens injected, staleness errors, latency and store size. One test goes further, and has a small local model answer from the selected notes (Section 6.17). We report no end-to-end task success. Recall under a budget is a necessary condition for those outcomes, not a measurement of them, and in the answer test it predicted answers poorly. A controlled study found that context strategy moved correctness on coding tasks by no more than 10 to 15 points [29], so large end-to-end gains should not be assumed.

2 Related work

Memory systems. Memory layers include OS-style paging by the agent [44], LLM-curated fact stores [13], temporal knowledge graphs with bi-temporal edge invalidation [50], linked notes [68], graph retrieval [16, 21, 22], and recursive summary trees [52]. Generative Agents scores memories by recency, importance and relevance [46]. Experience memories such as Reflexion, Voyager, ACE and ReasoningBank keep lessons or skills [43, 53, 64, 72]. All of these use a write path with LLM calls, a read path with a fixed k or an agent decision, or both. Of these systems, Zep/Graphiti is the one with a mechanism for validity. It is binary and depends on the extractor producing the invalidating edge.

Budgeted selection. Token-budgeted submodular selection for context is an active preprint line. PACMS applies facility location under a knapsack as a pluggable engine and lists learned relevance as future work [20]. AdaGReS scores relevance against redundancy [47], and PC-SubMax compresses prompts with a regularized objective [75]. The theory rests on Nemhauser et al. [42], on budgeted maximum coverage [30], on partial enumeration [18, 57], and on cost-benefit greedy with lazy evaluation [31, 35, 40]. MMR [9] and DPPs [32] target diversity without a budget in tokens.

Learning. Combinatorial and linear semi-bandits give regret with approximate oracles [10, 11, 34, 65], and linear submodular bandits extend this to diversified retrieval and to knapsacks [58, 69, 71]. Our analysis uses the confidence sets of OFUL [1]. In agent memory, DeMem proves regret bounds for a K-state memory partition [77] and MemRL shows Q-value convergence [73]; RSCB-MC and MERA use bandits to choose agent actions in coding [25, 26]. None of these treats token-budgeted selection of items whose validity changes. On reward signals, self-scored episodes inflate wrong answers [6], so we use execution signals.

Validity and forgetting. ACT-R base-level activation models the power law of forgetting [2, 3], and it has been applied to LLM memory [23]. Staleness is a measured failure: with a budget of two records, models acted on stale records in most episodes of one controlled study [41], and conflict benchmarks exist for chat [60]. Consolidation follows complementary learning systems [33, 39]. Rate–distortion views of compaction appear in [14, 54, 77]. Colaco and Lahjouji argue that repeated irreversible summarization degrades end-task error faster than retrieval-backed memory, which agrees with the context collapse reported for ACE [72], and motivates keeping sources.

Compression and parametric memory. Text compression such as LLMLingua [27, 28, 45] works with closed APIs and is complementary to selection. Gist tokens, AutoCompressors, ICAE, and KV-cache eviction [12, 19, 37, 48, 67, 74] and test-time weight updates [7, 8, 55] need access to model internals that closed APIs do not expose.

Benchmarks. LoCoMo [38], LongMemEval [66], MemoryAgentBench [24] and BEAM [61] cover conversational memory. SWE-ContextBench [76] and VibeMemBench [17] cover coding experience, MemoryCode covers multi-session coding instructions on synthetic data [49], and ContextBench covers retrieval within a task [36]. None of these, as far as we found, tests repository facts that change between sessions over real history.

3 A unified framework

Memory. At time t the store is a finite set M_t. An item is a tuple $m = (\text{text}, e, \tau^{\text{c}}, \tau^{\text{v}}, \pi, c, k, P)$: content, unit embedding, creation time, last time the item was confirmed, provenance, token cost $c(m) \in \mathbb{N}_{>0}$, an optional *slot key* k that identifies the fact the item states, and parent links P to the items it was consolidated from. Items sharing a key form a version chain.

Validity and usefulness. Fix a step context (query) q. Let $\rho_t(m) = \Pr[m \text{ still holds at } t]$ and $\mu_q(m) = \Pr[m \text{ helps with } q \mid m \text{ holds}]$. The *effective weight* of an item is $w_q(m) = \rho_t(m)\,\mu_q(m)$. Separating the two matters: ρ moves with time and changes in the world, μ with the query. Section 5 shows that a learner with no age or version information has linear regret.

Selection. Injecting $S \subseteq M_t$ has utility $U(S \mid q)$, and the context budget is a hard constraint:

$$\max_{S \subseteq M_t} \mathbb{E}\,[U(S \mid q_t)] \quad \text{s.t.} \quad c(S) := \sum_{m \in S} c(m) \le B. \tag{1}$$

We study the family

$$F(S) = (1-\gamma)\Big[\sum_{f \in \mathcal{F}_q} \omega_f \Big(1 - \prod_{m \in S} \big(1 - w(m)\, a_{mf}\big)\Big) + \eta \sum_{m \in S} w(m)\Big] + \gamma \sum_{c \in C} \frac{p_c}{\sum_{c'} p_{c'}} \max_{m \in S} s(c, m). \tag{2}$$

Here $\mathcal{F}_q$ is a set of facets of the query with weights $\omega_f \ge 0$, $\sum_f \omega_f = 1$, and $a_{mf} \in [0,1]$ is the chance that item m answers facet f given that it is valid and useful. The first term is the expected fraction of the need that gets covered when items succeed independently, the second the expected number of useful items injected, and the third a facility-location coverage of the candidate pool C with similarity $s \in [0,1]$ and fixed importance p_c, which removes near-duplicates.

Lemma 1. *For fixed $w \in [0,1]^M$, F is non-negative, monotone and submodular, with $F(\emptyset) = 0$. It is coordinate-wise non-decreasing in w and $|F(S;w) - F(S;w')| \le L \sum_{m \in S} |w(m) - w'(m)|$ with $L = (1-\gamma)(1+\eta)$.*

The online problem. A memory policy $\Pi = (\mathsf{W}, \mathsf{R}, \mathsf{C}, \mathsf{F})$ decides what to write, what to read, what to consolidate and what to forget. For a sequence of steps it earns

$$J(\Pi) = \mathbb{E} \sum_{t=1}^{T} \Big[U(S_t \mid q_t) - \kappa_{\text{r}}\, c(S_t)\Big] - \kappa_{\text{w}}\, \text{WriteCost}(\Pi) - \kappa_{\text{s}} \sum_t c(M_t), \tag{3}$$

subject to $c(S_t) \le B$ for all t. Reading costs tokens on every step, writing costs LLM calls or latency, and the store costs retrieval time and noise. Learning, in this paper, is updating Π and M_t; model weights never change.

3.1 Existing methods as special cases

Table 1 places the surveyed systems in (1)–(3). Nearly all read policies reduce to the modular case $\eta = 1$, $\gamma = 0$, $\mathcal{F}_q = \emptyset$ with a cardinality constraint and a method-specific score. The systems differ mostly in what they store, which is the W component.

Parametric methods (test-time training [55, 56], memory layers [8], Titans [7], gist tokens and KV-cache eviction [37, 48, 74]) keep memory in weights or attention state. They fall outside the framework by construction, and they require write access to weights or to the cache, which a closed API does not expose.

3.2 Where the special cases are suboptimal

The following statements are worst-case and elementary; the point is that each identifies a design defect that a better method must remove. Proofs are in Appendix A. Write OPT for the optimum of (1).

Proposition 1 (Cost blindness)**.** *Any rule that fills the budget in the order of a score independent of c (top-k fill, MMR, reciprocal rank fusion, reranking) has worst-case ratio at most $1/\lfloor B/c_{\min} \rfloor$ against* OPT*, even for additive U.*

Proposition 2 (Redundancy blindness)**.** *Any modular rule has worst-case ratio at most $1/k$ on coverage objectives with k facets when one facet has at least k near-duplicate items.*

Proposition 3 (Recency as proxy)**.** *A relevant item m followed by later items of total cost above B, each no larger than m, is never injected by recency-only selection, however relevant it is, whereas a rule that scores by relevance injects it whenever it outranks those items.*

Proposition 4 (Time-invariant scoring)**.** *Any selector whose score is invariant under permuting the timestamps of items with equal content features injects an outdated version ahead of the current one with probability at least $1/2$ on version pairs of equal text similarity to the query.*

Proposition 5 (Fixed policies)**.** *For any fixed linear scoring vector θ there is a $\theta^\star$ for which the fixed policy earns regret $\Omega(T)$, while the learner of Theorem 2 has regret $\tilde{O}(\sqrt{T})$ on the same instance.*

Table 1 shows which propositions apply: every k-limited row is cost-blind, every modular row is

Table 1: Surveyed methods as instances of the framework. "Score" is the w used by the read policy; k denotes a cardinality limit.

Method	Write W	Score w	Constraint	Validity ρ	Learns
Top-k RAG, hybrid, rerank [15, 51]	chunks	BM25/cosine/RRF/CE	k	none	no
MMR [9]	chunks	greedy rule, λ trade-off	k	none	no
PACMS, AdaGReS [20, 47]	chunks	facility location; relevance−redundancy	$c(S) \le B$	none	no
MemGPT/Letta [44]	agent tool calls	LLM decides paging	block sizes	none	no
Mem0 [13]	LLM add/update/delete	similarity	k	LLM rewrite	no
Zep/Graphiti [50]	LLM graph extraction	hybrid + rerank	k	$\rho \in \{0,1\}$ by edge invalidation	no
A-MEM [68]	LLM notes and links	similarity + links	k	LLM note evolution	no
HippoRAG 2 [22]	OpenIE triples	personalized PageRank	k	none	no
RAPTOR [52]	recursive summaries	similarity on all levels	k or tokens	none	no
Generative Agents [46]	observations, reflections	recency+importance+relevance	k	recency proxy	no
ACE, ReasoningBank [43, 72]	LLM-curated lessons	similarity or whole playbook	k	none	via text
Claude Code files [4]	model-decided notes	constant (query-independent)	200 lines/25 KB	none	no
Full history	everything	constant	window size	none	no
LLMLingua family [27, 45]	—	—	compresses $c(m)$	none	no

redundancy-blind, every row with no validity model or a recency proxy falls under Propositions 3–4, and every non-learning row under Proposition 5. The systems with invalidation (Zep/Graphiti) avoid Proposition 4 only for facts where the extractor produced the invalidating edge.

4 Method

Kanerva is the combination of the components below, implemented with SQLite, FTS5 and NumPy. No component calls an LLM. When a host agent is present it can supply structured writes through tools, which is how semantic extraction happens at no added API cost.

4.1 Write path

A write redacts secrets, deduplicates against live items (an identical write only refreshes the confirmation time τ^{v}), and stores the item with its FTS5 entry and embedding. A write with a key closes the version interval of the live item with that key: the old item keeps its text and gets τ^{end} and a pointer to its successor. Nothing is rewritten, so history stays queryable.

Most notes come without a key, and an agent cannot see which older note a new one replaces. So a write through the `memory_remember` tool returns up to three live facts of the same scope that look like the same subject: those with at least half of the two notes' *idf*-weighted word mass in common, with enough rare words in common, or with vector cosine above 0.95. Notes that differ in length by more than a factor of two are skipped. This only lists candidates. The host agent decides, and `memory_forget` with `replaced_by` then closes the old note the way a key chain does. Nothing in the memory layer calls a model for this.

4.2 Read path

Algorithm 1 shows retrieval. Candidates are the union of the top-200 lexical and dense hits, plus the items written next to the 50 best fused hits in the same session. Neighbours matter because a conversation often splits one fact across adjacent turns, and on our development data they added about twice as much evidence per candidate as a larger pool.

Relevance. Each candidate gets a calibrated probability of being needed, from gradient-boosted trees over 29 features in five groups: retrieval signals (ranks, scores, query-term coverage), the item itself (length, content-word density, question form, numbers), role and speaker (whether a first-person query meets a user turn, whether a named speaker appears in the query), query–item overlap (entities, numbers, and a question type such as "when" or "how many" against dates and numbers in the item), and context (similarity of neighbouring turns and of the session, recency, ACT-R base-level activation [3]). Three more features come from an optional local reranker. The trees are fitted on LongMemEval-S, LoCoMo, BEAM-1M and 34 RepoMem repositories, keyed and unkeyed, with conversations and code as two families of equal weight and each query equal within its set. They are exported to a compact array form and evaluated with NumPy in tens of milliseconds for a few hundred candidates, so the package has no learning library at run time. Separate models cover the lexical, static-embedding, dense and reranker configurations. The static model2vec model keeps its earlier fit and was not refitted on the 34 repositories.

Objective. With calibrated weights, $F(S) = \sum_{m \in S} w_m$ is the expected number of needed items in S. This is the modular case of the objective of Section 3, so Theorem 1 still applies, and Proposition 7 says how probability error reaches the selection. With calibrated weights the coverage facets and the facility-location term had no measurable effect on development data, so they are off by default. Length enters twice, as a feature and as the cost, which is intended: the model can learn that short turns are rarely needed, so density

Algorithm 1 RETRIEVE(q, B, t)

$C \leftarrow$ top-N of FTS5 $\cup$ top-N of dense search $\cup$ same-session neighbours of the best fused hits
Remove superseded items, and items with a newer near-twin that covers q at least as well, unless q asks for history
for $m \in C$ **do**
    $p_m \leftarrow$ relevance model on features of (m, q); $x_m \leftarrow (p_m, u_m)$ ▷ u_m: past use
    $\mu_m \leftarrow \mathrm{clip}_{[0,1]}\big(x_m^\top \hat{\theta} + \beta \|x_m\|_{V^{-1}}\big)$
    $\rho_m \leftarrow$ Lomax survival (4) at $t - \tau^{\mathrm{v}}(m)$
    $\rho_m \leftarrow \rho_m \prod_{m' \text{ newer, covers } q} \big(1 - \pi\, g(s(m, m'))\big)$ ▷ no key
    $w_m \leftarrow \rho_m \mu_m$
end for
$\mathcal{S} \leftarrow \{\text{GREEDY}(F, B, \varrho) : \varrho \in \{1, \frac{1}{2}\}\} \cup \{\arg\max_{c(m) \le B} F(\{m\})\}$
return $\arg\max_{S \in \mathcal{S}} F(S)$, and on request certificate (5)

Algorithm 2 GREEDY(F, B, ϱ)

$S \leftarrow \emptyset$; $A \leftarrow \{m : c(m) \le B\}$
while $A \neq \emptyset$ **do**
    $m \leftarrow \arg\max_{m \in A} \left[F(S \cup \{m\}) - F(S)\right] / c(m)^{\varrho}$; remove m from A
    if the gain is 0 **then break**
    end if
    if $c(S) + c(m) \le B$ **then** $S \leftarrow S \cup \{m\}$
    end if
end while
return S

no longer favours short junk once the strong candidates run out. This removes the main weakness of the first version of the method, a linear prior over retrieval signals alone, which lost to plain ranking at large budgets.

Only the density variant $\varrho = 1$ and the singleton carry the guarantee; the other runs can only raise F, because the maximum is taken over all runs.

The plugins add steps that the recall benchmarks do not exercise. They drop selected items whose probability is below 0.02. Before that, a *query gate* decides whether the store can answer the prompt at all, since selection alone fills the budget with the best of a bad lot. The gate is a logistic model over six signals of the whole result list: the best candidate's *idf*-weighted term cover, the number of query terms, the best relevance weight, the spread of lexical scores, the best cosine, and that cosine's gap to the mean of the top 50. A store without vectors uses the first four. The model is fitted to tell a store's own questions from questions drawn from other stores, and its threshold is set to keep 0.97 of own recall on the data it was fitted on. Below the threshold the prompt gets no notes. Session start is not gated, because its query is generic.

File notes. A prompt says little about a change, while the file an agent opens says a lot. When a session first touches a file, by reading or editing it, the hook returns live notes that name that file, newest first, within 150 tokens, and the session as a whole stays within eight times that. A name that many projects share, such as `index.ts`, counts only together with its folder, and the notes the edit hook writes by itself are skipped. This is a plain lookup by name with no relevance model. Adding the session's files to the prompt query was tested instead, and dropped (Section 6.12).

4.3 Validity

For a class k of facts (for example, function signatures), the unknown change rate has a Gamma(α, β) prior. After n_k observed changes over E_k item-days of exposure, the posterior predictive probability that an item confirmed a days ago still holds is

$$\hat{\rho}_k(a) = \left(1 + \frac{a}{\beta + E_k}\right)^{-(\alpha + n_k)}. \tag{4}$$

The class hazards are estimated from the store's own version history, so stable classes stop decaying without any tuning. (4) is a Lomax tail, which is a power law for ages well beyond $\beta + E_k$; at smaller ages it behaves like an exponential. So the mixture argument behind this estimator gives the same tail as the power-law decay seen in human memory and in the environment [2]; see Proposition 10.

Keyed facts have exact version chains: a superseded item is dropped from the candidates, and a query with historical intent ("previously", "used to") brings old versions back with $\rho = 1$. Text without keys has no such chain. For it we use a heuristic: a newer item with similarity above 0.8 discounts an older one (the product in Algorithm 1), and a newer near-twin above 0.97 removes it. Without embeddings, the similarity comes from feature hashing, which rates two versions of one fact lower than a sentence model does, so the removal threshold there is 0.85. A newer twin stands in for an older item only if it covers the query at least as well, measured by *idf*-weighted term cover, because facts about two neighbouring functions read alike and a question names one of them. Only facts replace each other, since two similar events are still two events. This rule fires only on near-identical text, so it suits structured notes and rarely fires on natural-language updates; Section 6 measures both cases.

4.4 Learning

The online layer sits on top of the frozen relevance model. It keeps a ridge estimate over three features,

a bias, the model's probability and the item's past use, starting from $\theta_0 = (0, 1, 0)$, so that with no feedback it returns the model's probability: $\hat{\theta}_t = \theta_0 + V_t^{-1} \sum_{(x,y)} (y - x^\top \theta_0) x$ with $V_t = \lambda I + \sum x x^\top$. The bias is used in fitting and zeroed when scoring. It absorbs the base rate of the reward, including false positives from a noisy signal; a constant added to every item would otherwise push the density rule toward short items. Feedback $y \in [0, 1]$ says whether an injected item was used, and a used item counts as reconfirmed. Items known to be invalid at feedback time are skipped, since they say nothing about μ. The update is $O(d^2)$ by Sherman–Morrison per item and works offline. In the plugin, an injected item counts as used when its identifiers or paths appear in later tool calls or edits, and a failed test halves the reward, following the evidence that self-reports inflate reward [6]. Item-level counters always update.

The plugins do not use this bandit. With `KANERVA_LEARN=1` they scale each note by its own record instead, an *item lift*: with u_m observed uses and e_m the sum of the model's probabilities over the times the note was injected, μ_m becomes $\mathrm{clip}_{[0,1]}\big(\mu_m (u_m + \kappa)/(e_m + \kappa)\big)$, with $\kappa = 5$. A note with no record keeps the model's probability, and one that helps more often than the model expected rises. This asks only about the item, so it needs no weights over shared features. It is opt-in, because Section 6 measures it on simulated feedback only, and the theory of Section 5 covers the bandit and not this rule.

4.5 Consolidation and forgetting

Consolidation is reversible. Star clusters around a leader, with every member above a similarity threshold of 0.9 by default, are summarized extractively: density greedy on a sentence-level facility location with rate R, which yields the rate–distortion guarantee of Theorem 3. The summary is a new semantic item with parent links, the sources move to an archive tier, and EXPAND restores them on demand. This follows the evidence that repeated irreversible rewriting erodes detail [14, 72]. Forgetting moves items to the archive when they have been idle for τ days or when the active store exceeds a token capacity, evicting the lowest $\rho \cdot \text{activation} \cdot \text{usefulness}/c$ first. Both rules are opt-in, because Section 6 finds that they cost recall. Deletion happens only on an explicit user command.

4.6 Multi-resolution

An item above 160 tokens can also enter selection as an excerpt: the run of consecutive sentences within 128 tokens that covers the most *idf*-weighted query terms, marked with the item's id. The excerpt's weight is the item's probability times the share of query weight it keeps, or its share of the tokens if that is larger, and its cost is its own. The two versions of an item are mutually exclusive. That adds a partition constraint to the knapsack, which Theorem 1 does not cover, so excerpts are a heuristic. The agent pulls the full text with EXPAND when it needs it. This mode is off by default.

4.7 Complexity

With N active items, n candidates, embedding dimension D, $\mathcal{T}$ trees of depth at most h, and $|S| = k$ selected: a write costs $O(\log N)$ plus one embedding; candidate generation costs $O(ND)$ by brute force (or sub-linear with an ANN index) plus the FTS5 query; scoring costs $O(n\mathcal{T}h)$; on the modular objective, greedy reduces to a sort and costs $O(n \log n)$ per run. A bandit update costs $O(kd^2)$. Space is $O(N(D+\bar{c}))$. The memory layer itself uses no LLM tokens; the only token cost is the injected $c(S) \le B$.

5 Theory

All proofs are in Appendix A. Throughout, F is any set function with the properties of Lemma 1.

5.1 Selection

Theorem 1 (Selection guarantee)**.** *Let F be non-negative monotone submodular with $F(\emptyset) = 0$ and let $c(m) \le B$ for all candidates. The set returned by Algorithm 1 satisfies $F(S) \ge \frac{1}{2}(1 - \frac{1}{e})\,\mathrm{OPT}$.*

This is the classical cost-benefit argument [31, 35], restated with a full proof because the constant in related analyses has needed correction [59]. Better constants exist for this algorithm family, 0.405 [59], and $1 - 1/e$ with partial enumeration at higher cost [18, 57]. The worst case is pessimistic for a practical reason: at each query the following bound can be computed.

Proposition 6 (Per-query certificate)**.** *For any feasible S, with $\Delta(m \mid S) = F(S \cup \{m\}) - F(S)$,*

$$\mathrm{OPT} \le F(S) + \max_{\substack{y \in [0,1]^M \\ \sum_m y_m c(m) \le B}} \sum_{m \notin S} y_m \, \Delta(m \mid S), \quad (5)$$

and the right-hand side is computed by sorting on Δ/c in $O(n \log n)$.

5.2 Selection with estimated probabilities

With the relevance model of Section 4, the objective is modular: $F(S) = \sum_{m \in S} w_m$, the expected number of needed items in S when w_m is the probability that m

is needed. The model only estimates these probabilities, so the question is how estimation error reaches the selection.

Proposition 7 (From probability error to selection loss)**.** *Let p_m be the probability that candidate m is needed and $\hat{p}_m \geq 0$ an estimate, and write $e(S) = \sum_{m\in S} |p_m - \hat{p}_m|$. Let $S^\star$ maximise $\sum_{m\in S} p_m$ under the budget. If $\hat{S}$ is feasible and $\sum_{m\in\hat{S}} \hat{p}_m \geq \alpha \max_{c(S)\leq B} \sum_{m\in S} \hat{p}_m$, then*

$$\sum_{m\in\hat{S}} p_m \;\geq\; \alpha \sum_{m\in S^\star} p_m - \alpha\, e(S^\star) - e(\hat{S}).$$

Density greedy with the best singleton has $\alpha = \frac{1}{2}$ on modular objectives, and an exact knapsack has $\alpha = 1$, in which case the loss is at most $e(S^\star \triangle \hat{S})$.

So only the error on selected and optimal items matters, not the error on the whole pool. The worst case separates the two algorithms by a factor of two, while on our dev data they differ by about a point, which is why the simpler one is kept.

5.3 Learning with expiring items

The closest results [58, 69, 71] assume that an item's expected value is a stationary linear function of its features. Memory violates this: a fact's value collapses when the fact stops holding, while its text, and so its features, stay the same. We therefore model the two factors separately.

Setting. In round $t = 1, \ldots, T$ the learner sees candidates C_t with features $\|x_{t,m}\|_2 \leq 1$ (rescaled by a constant if needed), costs, and a function $F_t(S; w)$ satisfying Lemma 1 with constant L. There is an unknown $\theta^\star$ with $\|\theta^\star - \theta_0\|_2 \leq \Sigma$ and $\mu_{t,m} = x_{t,m}^\top \theta^\star \in [0,1]$. Each item is valid with probability $\rho_{t,m} \in [0,1]$ given the history before round t, which may vary arbitrarily with t, and the true weight is $w^\star_{t,m} = \rho_{t,m}\mu_{t,m}$. For each injected item the learner observes $Z_{t,m} \in \{0,1\}$ with mean $\rho_{t,m}$, possibly correlated within a round, and, if $Z_{t,m} = 1$, a reward $y_{t,m} = \mu_{t,m} + \xi_{t,m}$, where $\xi_{t,m}$ is R-sub-Gaussian given the past, the current Z and the earlier rewards of the round. At most K items are injected per round. A validity estimate satisfies $|\hat{\rho}_{t,m} - \rho_{t,m}| \leq \varepsilon_{t,m}$ for all t, m with probability at least $1 - \delta/2$.

Algorithm. Use $\bar{w}_{t,m} = \min(1, \hat{\rho}_{t,m} + \varepsilon_{t,m}) \cdot \mathrm{clip}_{[0,1]}(x_{t,m}^\top \hat{\theta}_t + \beta_t \|x_{t,m}\|_{V_t^{-1}})$, with $\hat{\theta}_t, V_t$ the prior-centred ridge estimates over the rounds where $Z = 1$, and pick S_t with an α-approximate oracle for $F_t(\cdot\,; \bar{w}_t)$ under the budget.

Theorem 2 (Validity-aware regret)**.** *Let $\lambda \geq K$ and $\beta_t = \beta_T := R\sqrt{d \log\big(4(1 + KT/\lambda)/\delta\big)} + \sqrt{\lambda}\,\Sigma$. With probability at least $1 - \delta$,*

$$\sum_{t=1}^{T} \Big[\alpha \max_{c(S)\leq B} F_t(S; w_t^\star) - F_t(S_t; w_t^\star)\Big] \leq L\Bigg[2\beta_T \sqrt{2dKT \log\Big(1 + \frac{KT}{d\lambda}\Big)} + K\sqrt{2T \log\frac{4}{\delta}} + 2\sum_{t=1}^{T} \sum_{m\in S_t} \varepsilon_{t,m}\Bigg].$$

The comparator is the per-round optimum under the time-varying $w_t^\star$, so the bound is dynamic, and the $\rho_{t,m}$ need no variation budget: the burden moves into estimating them, through the last term. The first term is the usual linear bandit price, the second comes from the randomness of validity, and warm starting helps through Σ. With $\alpha = \frac{1}{2}(1 - \frac{1}{e})$ from Theorem 1 and $L = (1-\gamma)(1+\eta)$, the theorem covers Algorithm 1 run with β_T and the margin ε.

What this does and does not say. The theorem describes how regret scales, and it is not a practical guarantee. Since $F \leq 1$ in the original objective, regret is trivially at most T, and with $K = 40$, $R = \frac{1}{2}$, $\Sigma = 1$, $\delta = 0.05$ and $L = 1$, the bound falls below T near 1.2×10^7 rounds at $d = 12$. Letting the online layer see only three features, with the frozen model's probability as one of them, moves that point to 1.6×10^6 rounds, which is better and still far from a session count. The shipped defaults use $\beta = 0$ and no margin, and they use a heuristic rule on top of (4), so the code is not the analyzed algorithm. The recall tables in Section 6 use the static model and do not rest on online learning at all.

What it covers. The theorem analyses the ridge bandit of Section 4, which the library keeps. The opt-in learning path of the plugins is the item lift of that section, a different rule. This analysis does not cover it.

Proposition 8 (Validity error of the hazard estimate)**.** *Let items of class k have lifetimes that are $\mathrm{Exp}(\lambda_k)$, with right-censoring that depends only on the past. Let N be the number of observed changes, E the total exposure up to a stopping time, and $\hat{\lambda}_k = N/E$. If $E \geq E_0$ almost surely and $\eta := 2\sqrt{\log(2/\delta)/(\lambda_k E_0)} \leq \frac{1}{2}$, then with probability at least $1 - \delta$, for all ages $a \geq 0$ simultaneously,*

$$|e^{-\hat{\lambda}_k a} - e^{-\lambda_k a}| \leq \frac{2}{e}\,\eta.$$

The bound does not depend on age: the error is governed by $\lambda_k E_0$, the expected number of observed changes. This estimator counts censored exposure, as

the code does, whose posterior (4) has the same limit. If every class in use accumulates exposure as $\lambda_k E_0 \geq \nu t$, then $\varepsilon_{t,m} = \tilde{O}(1/\sqrt{\nu t})$ by a union bound over a grid of times, $\sum_t \sum_{m \in S_t} \varepsilon_{t,m} = \tilde{O}(K\sqrt{T/\nu})$, and the total is $\tilde{O}((d\sqrt{K} + K/\sqrt{\nu})\sqrt{T})$. For the conditional probabilities of the setting to match this survival curve, age must restart when an item is confirmed; the code does this for items that are used.

Proposition 9 (Validity-blind learners)**.** *Let $m \geq 2$. There is an instance in which m items per round have identical features, exactly one is valid, and B fits one item. Any learner that does not observe age or version information has $\mathbb{E}[\sum_t \alpha\,\mathrm{OPT}_t - F_t(S_t)] \geq (\alpha - \frac{1}{m})T$ for every $\alpha > 1/m$. So it has linear regret, for all horizons, whereas Theorem 2 applies with $\varepsilon \equiv 0$ if the version chain is known.*

5.4 Validity and activation

Proposition 10 (Power-law survival from a hazard mixture)**.** *If hazards in a class are* $\mathrm{Gamma}(\alpha, \beta)$ *distributed, the population survival is $S(a) = (1 + a/\beta)^{-\alpha}$. As $a \to \infty$, $\log S(a) = -\alpha \log a + O(1)$, the form of ACT-R single-trace base-level activation with decay $d = \alpha$.*

5.5 Consolidation

Let X be a cluster of sentences with weights p ($\sum_x p_x = 1$), costs c, cosine similarities $s \in [0, 1]$ with $s(x, x) = 1$, and let $\Phi(Z) = \sum_x p_x \max_{z \in Z} s(x, z)$, $D(Z) = 1 - \Phi(Z)$ be the distortion of a summary $Z \subseteq X$.

Theorem 3 (Rate–distortion for extractive consolidation)**.** *Let Z_ℓ be the first ℓ picks of density greedy on Φ, with rate $R_\ell = c(Z_\ell)$. For every summary $Z^\star$ of rate $R^\star$,*

$$D(Z_\ell) \leq e^{-R_\ell/R^\star} + \big(1 - e^{-R_\ell/R^\star}\big) D(Z^\star).$$

Moreover, for every query vector e_q and every ℓ_{u}-Lipschitz item score $u_q(x) = \phi(\langle e_q, e_x \rangle)$,

$$\mathbb{E}_{x \sim p}\big[u_q(x) - u_q(z(x))\big] \leq \ell_{\mathrm{u}} \sqrt{2\, D(Z_\ell)},$$
$$z(x) = \arg\max_{z \in Z_\ell} s(x, z).$$

The token saving is $c(X) - R_\ell$, so distortion decays exponentially in rate against any reference summary. The statement covers the greedy prefix; the implementation continues past the first sentence that does not fit, as a heuristic. The bound averages over p and says nothing about a rare needle in the cluster. That is why consolidation here keeps the sources: if the summary is retrieved, Expand recovers any source at the cost of its tokens.

5.6 Store size

Theorem 4 (Store stability)**.** *Let items arrive as a Poisson process of rate a, and let each be accessed by an independent Poisson process of rate r, drawn i.i.d. from a distribution with $\mathbb{E}[(e^{r\tau} - 1)/r] < \infty$. An item is archived after an idle gap of τ. Then an item's active lifetime has mean $(e^{r\tau} - 1)/r$ given r, and the number of active items converges in distribution to* $\mathrm{Poisson}\big(a\,\mathbb{E}[(e^{r\tau} - 1)/r]\big)$.

Without forgetting, the store grows linearly in t. With an idle rule, the active store is bounded in distribution, and the retained set is biased exponentially toward frequently used items. The theorem says nothing about what forgetting costs in recall, and it assumes independent access processes, whereas real access patterns are bursty. Section 6 measures both.

6 Experiments

Sections 6.2 to 6.5 are the record of the method before version 0.5.0: its development results and its first two held-out tests. They are kept as they were run, and their models, thresholds and lexical rows are not those of the current release. The current release is measured in Section 6.6 and the sections after it, up to and including Section 6.16.

6.1 Setup

Mode. No LLM takes part in the method. Metrics are retrieval-level, so we report what reaches the context and not what a model does with it, except in Section 6.17, where a small local model answers from the selected notes.

Development data. *LongMemEval-S* [66] has 500 questions, each with its own store of about 490 turns. *LoCoMo* [38] has 10 long conversations. *BEAM-1M* [61] has 35 long conversations of about 2,000 turns. *RepoMem* (new) turns eight Python repositories into memory streams, at 14 snapshots over six years. Each interval is a session in which a simulated agent records commit notes and facts observed in changed files. Facts (constants, signatures, symbol locations, dependency constraints) come from `ast`, so a fact is stale exactly when the same key held another value at an earlier snapshot. Fact queries ask for the current value; task queries use a real later commit subject, with earlier commits on the same files as gold. RepoMem runs with slot keys, as a host agent would supply, and without.

First held-out test. *BEAM-100K and BEAM-500K*: 984 questions in 55 conversations that share no seed with BEAM-1M, keeping every question with labelled evidence. Their turns average 232 tokens for

evidence and 482 for the rest, and the full history of a conversation averages 379,187 tokens. *RepoMem* on five repositories outside development (marshmallow, pluggy, packaging, loguru, tqdm): 1,248 queries over three seeded streams each. No model, threshold or design choice was made on this data. The method was frozen first, and the test ran once, with a single script.

Baselines. Recency, BM25 (FTS5), dense (all-MiniLM-L6-v2 [62]), hybrid RRF [15], hybrid followed by a cross-encoder (ms-marco-MiniLM-L-6-v2) on the top 50, MMR [9], Generative Agents scoring [46] with constant importance, facility location under a knapsack as in PACMS [20], hybrid with superseded versions removed (temporal invalidation alone, in the spirit of [50]), hybrid listed newest first, and hybrid keeping only the newest of near-twins. We also keep the *first version* of the method: a linear prior over retrieval signals with no item, role or context features, and a pool of 100 from each retriever. All methods share candidate generators, token counts (`cl100k_base`) and a 14-token per-item overhead, and every ranking is filled greedily to the budget, which already makes the baselines budget-aware. Systems that need an LLM on write were not run.

Statistics. The independent unit is a conversation on BEAM (55) and a repository on RepoMem (5 in the first test), since queries within a conversation are dependent and seeds repeat RepoMem queries. We give bootstrap intervals on the unit, and p-values from two-sided sign-flip permutation tests on the unit means, with Holm correction across budgets within a comparison. With five units the smallest possible two-sided p is 1/16, so no RepoMem comparison can reach significance. There, read the interval and the count of units favouring each side, and note that an interval containing zero on five units is weak evidence that two methods are equal.

6.2 Development results of the earlier version

Table 2 gives cross-fitted recall: each query is scored by a model that never saw its question, conversation or repository. The relevance model raises mean recall over the first version on all four datasets, from 0.751 to 0.801 on LongMemEval-S, 0.663 to 0.732 on LoCoMo, 0.863 to 0.881 on RepoMem and 0.344 to 0.457 on BEAM-1M. The large-budget loss to hybrid ranking disappears on LoCoMo: 0.920 against 0.861 at 4096 tokens. Without a cross-encoder the method still trails the reranker at the smallest budget, 0.454 against 0.517 on LoCoMo at 128 tokens, because at that size one item decides everything and only joint reading of query and text separates the top candidates. With cross-encoder features it closes that gap, at 0.530.

The candidate pool matters most on BEAM, where a question has many evidence turns. There the top 100 from each retriever reach 0.665 of the evidence, and 200 reach 0.744, at a cost of 326 candidates instead of 167. Adding the neighbours of the 50 best hits reaches 0.731 with 225 candidates, so neighbours add about twice as much evidence per candidate as a deeper pool, while pseudo-relevance feedback adds little (0.677).

Table 3 removes one feature group at a time and refits. Item features and neighbourhood features carry most of the gain, and query–item overlap adds little on top of them.

The cross-fitted probabilities are close to calibrated: over ten equal-count bins the expected calibration error is 0.0005 on LongMemEval-S, 0.0003 on LoCoMo, 0.0016 on RepoMem and 0.0017 on BEAM-1M, against base rates near one percent. They are somewhat overconfident at the top on BEAM-1M, where the highest-scored one percent of candidates are predicted at 0.349 and observed at 0.287.

With such weights, the choice of knapsack solver hardly matters: over the four datasets, mean recall is 0.801 for density greedy against 0.800 for an exact knapsack on LongMemEval-S, and 0.457 against 0.454 on BEAM-1M. Cost still matters where item lengths vary: ranking by probability alone falls to 0.431 on BEAM-1M. We keep density greedy because Theorem 1 covers it.

Table 2: Cross-fitted evidence recall on development data. Mean is over the six budgets.

Method	128	256	512	1024	2048	4096	mean
LongMemEval-S							
First version	0.406	0.641	0.770	0.853	0.903	0.935	0.751
Hybrid RRF	0.379	0.593	0.669	0.756	0.848	0.907	0.692
Hybrid + reranker	0.442	0.695	0.783	0.860	0.906	0.933	0.770
BM25	0.392	0.611	0.675	0.743	0.811	0.861	0.682
Kanerva	0.407	0.673	0.845	0.926	0.970	0.986	0.801
Kanerva + reranker	0.434	0.700	0.866	0.931	0.970	0.985	0.814
Kanerva, lexical	0.383	0.661	0.814	0.893	0.928	0.941	0.770
Kanerva, static embedder	0.402	0.684	0.845	0.910	0.964	0.986	0.799
LoCoMo							
First version	0.408	0.555	0.661	0.733	0.785	0.837	0.663
Hybrid RRF	0.335	0.476	0.592	0.690	0.788	0.861	0.624
Hybrid + reranker	0.517	0.631	0.696	0.733	0.791	0.861	0.705
BM25	0.410	0.548	0.630	0.699	0.753	0.799	0.640
Kanerva	0.454	0.617	0.722	0.808	0.870	0.920	0.732
Kanerva + reranker	0.530	0.662	0.758	0.824	0.873	0.924	0.762
Kanerva, lexical	0.441	0.568	0.654	0.729	0.776	0.813	0.663
Kanerva, static embedder	0.462	0.611	0.716	0.797	0.861	0.907	0.725
RepoMem							
First version	0.771	0.824	0.858	0.881	0.903	0.940	0.863
Hybrid RRF	0.777	0.818	0.859	0.886	0.921	0.946	0.868
Hybrid + reranker	0.808	0.851	0.875	0.894	0.922	0.946	0.882
BM25	0.717	0.767	0.818	0.857	0.885	0.907	0.825
Kanerva	0.802	0.839	0.871	0.897	0.926	0.954	0.881
Kanerva + reranker	0.812	0.847	0.875	0.901	0.930	0.956	0.887
Kanerva, lexical	0.722	0.776	0.840	0.875	0.897	0.910	0.837
Kanerva, static embedder	0.722	0.782	0.833	0.872	0.919	0.949	0.846
BEAM-1M							
First version	0.193	0.288	0.363	0.364	0.404	0.450	0.344
Hybrid RRF	0.198	0.303	0.372	0.309	0.340	0.368	0.315
Hybrid + reranker	0.218	0.319	0.391	0.351	0.385	0.425	0.348
BM25	0.165	0.256	0.291	0.241	0.263	0.296	0.252
Kanerva	0.212	0.333	0.438	0.518	0.593	0.651	0.457
Kanerva + reranker	0.217	0.347	0.442	0.524	0.592	0.652	0.462
Kanerva, lexical	0.181	0.276	0.352	0.411	0.448	0.476	0.357
Kanerva, static embedder	0.185	0.295	0.388	0.481	0.543	0.605	0.416

Table 3: Mean recall lost when a feature group is removed and the model refitted, on development data. Positive means the group helps.

Group removed	LongMemEval-S	LoCoMo	RepoMem	BEAM-1M
Item	+0.008	+0.032	+0.010	+0.008
Role and speaker	+0.003	+0.011	-0.000	+0.010
Query–item overlap	+0.001	-0.002	+0.001	+0.005
Neighbourhood	+0.008	+0.033	+0.001	+0.009
Recency and activation	+0.001	-0.004	+0.002	+0.010

6.3 First held-out test, earlier version

Table 4 gives the results, and Table 5 the paired differences.

Long conversations. On BEAM the method pulls clearly ahead of every rival from 1024 tokens, and the lead grows with the budget. Averaged over budgets it has 0.566 against 0.478 for the reranker and 0.440 for hybrid, a difference of +0.088 over the reranker with unit interval [+0.072, +0.102], and 52/55 conversations favour it. At 1024 tokens recall is 0.635 against 0.488. Below 1024 tokens the difference from the reranker is not significant, and at 128 tokens the two have 0.324 and 0.322. Adding cross-encoder features gives 0.350 at 128 tokens and 0.644 at 1024. The gain over the first version is +0.086 with interval [+0.074, +0.098]. Removing the wide pool and neighbours costs +0.016, and removing cost density costs +0.016 (Table 6).

Code history. On RepoMem the picture is mixed. The method leads at the smallest budget, with keys 0.782 against 0.726 for the reranker and 0.764 for hybrid with invalidation. From 1024 tokens it falls behind: at 4096 tokens with keys it has 0.926 against 0.952, and without keys 0.917 against 0.951. Averaged over budgets the difference to hybrid is +0.007 with keys and -0.009 without, with intervals [-0.024, +0.028] and [-0.041, +0.017] that contain zero on five repositories. The first version is beaten on every repository, by +0.015 without keys. Most of the loss at large budgets sits on the task track, and it comes from the near-twin rules, which take look-alike commit notes for versions of one fact. Without keys at 4096 tokens, task recall is 0.642, against 0.788 with those rules removed and 0.761 for hybrid, while fact recall is 0.988 against 1.000, which accounts for the rest. Removing the rules altogether would bring back the stale injections shown next, so we restricted them to facts and ran a second held-out test (Section 6.5).

Lighter configurations. The lexical model needs no embeddings, and it was the plugin default at the time. Its rows in this section and the next two were computed on stores that held vectors, so they compared near-twins by vectors, which a real lexical store cannot do (Section 6.6). On BEAM it has 0.488 against 0.374 for BM25, on the same candidates. On RepoMem without keys it has 0.809 against 0.818, and it loses to BM25 at 4096 tokens, 0.895 against 0.939. With the static model2vec embedder [63], which needs no neural runtime, the method reaches 0.536 on BEAM, better than the lexical model, but 0.779 on RepoMem, worse than the lexical model and worse than BM25. On changed RepoMem facts it also injects a stale version more often, 0.371 against 0.206. So the configurations the plugins ran at the time were strong on conversations and not on code facts: the numbers in the first paragraphs of this section need the MiniLM embedder.

Mem0. On a random 40-question development sample of LongMemEval-S, Mem0's verbatim mode reached 0.829 at 1024 tokens against 0.877 for the first version of the method and 0.846 for hybrid on the same questions, with 9 ms per write. Mem0's LLM extraction, its main feature, was not exercised.

6.4 Staleness in the first test

Table 7 reports, on RepoMem queries whose fact changed, how often an outdated version is *injected*, and the *error*: how often it is injected and the current version is absent or listed below it. Similarity retrieval injects the outdated version nearly always, 0.997 of the time at 1024 tokens. With keys, exact version chains remove stale items entirely, and hybrid with invalidation does this as well as Kanerva. Without keys, Kanerva injects a stale version 0.194 of the time, and the simple rule of keeping the newest near-twin reaches 0.227. So on RepoMem the gain comes from that rule, which Kanerva also uses, and not from its probabilistic validity model. We have no ablation that isolates the hazard model's value.

The rule depends on near-identical text, and RepoMem's versions differ only in a value. On the *natural-language* updates of LongMemEval, evidence for a fact and for its update are not near-twins, the rule does not fire, and on development data Kanerva injects the outdated turn on 1.000 of questions against 0.857 for hybrid, since it finds more evidence of both kinds. Listing items newest first keeps the error to 0.029, against 0.471 for hybrid in rank order. The store alone cannot tell that a free-text statement replaces an older one, since that needs semantics an LLM-free method lacks. A host agent has them, and Section 6.13 measures one.

6.5 Second held-out test, earlier version

The first test showed that the near-twin rules treat similar events as versions of one fact. We changed one thing: an item can only be replaced by, or discounted against, another fact, so episodic notes such as commits never replace each other. Thresholds were tuned

on the seventeen repositories seen so far, choosing the highest recall that did not raise stale injection, which kept them unchanged. The relevance model was not refitted. On BEAM all items are episodic, so the rule is inactive there, and the shipped method matches the variant without near-twin rules in Table 6, a difference of -0.003.

Eight more repositories, named before their histories were fetched, form the test: uvicorn, anyio, trio, tox, waitress, gunicorn, alembic and jsonschema, with 2,014 queries. Eight units also make significance reachable. The method was frozen first, and the test ran once. Tables 8 and 9 give the results.

The change helps on its own: averaged over budgets it adds +0.011 with keys and +0.010 without, with unit intervals [+0.005, +0.017] and [+0.004, +0.016]. Without keys at 4096 tokens, task recall rises from 0.685 to 0.786, against 0.705 for hybrid, and stale injection stays at 0.243 against 0.244 for the earlier rule and 0.991 for hybrid.

With keys, the method now beats every rival: +0.021 over hybrid, +0.018 over the reranker and +0.010 over hybrid with invalidation, each with $p = 0.008$ on the mean, and 8/8 repositories favour it over hybrid. Without keys it does not: the difference to hybrid is -0.004 with interval [-0.023, +0.010], and to the reranker -0.008 with interval [-0.028, +0.005]. It leads at 128 tokens and trails slightly at 1024. Part of that is the rule still firing on exact facts: fact recall at 4096 tokens is 0.975 against 0.997.

The lighter configurations remain behind on code. The lexical model without keys is level with BM25 on average (-0.003, interval [-0.015, +0.006]), and below it at 1024 tokens, on 1/8 repositories in its favour. The static embedder injects a stale version 0.374 of the time.

We tried to narrow that gap by comparing twins with hashed words, as the lexical model does, instead of the static embedder's vectors. Every RepoMem repository has been seen by now, so these are development numbers. A setting had to lower stale injection and keep mean recall on all four groups of repositories, and none of the 5 we tried did. The closest raised mean recall on each group, by +0.039 on the second test's repositories (interval [+0.018, +0.063]), but stale injection there rose from 0.374 to 0.495. Stricter thresholds lowered it and lost recall. We kept the embedder's vectors.

Beyond Python. RepoMem's task track needs only commit history, so unlike the fact track it can run on other languages. Nine active repositories, named before their histories were fetched, give 590 task queries: vite, hono and zod in TypeScript, gin, caddy and fzf in Go, ripgrep, axum and clap in Rust. Nothing was refitted, and the models had seen code only from Python repositories. Recall stays in the same range as on the Python task track, 0.506 against 0.541 at 1024 tokens, so the method does not break. Its advantage does not carry over, though. On the second test's task track it led hybrid search by +0.049 at 1024 tokens. Here it is level over all budgets (-0.006, interval [-0.015, +0.003], 2/9 repositories in its favour) and behind at 128 and 256 tokens (-0.012, interval [-0.020, -0.005], and -0.013, interval [-0.019, -0.006]). It is level with the reranker (+0.012, interval [-0.011, +0.033]), and the lexical model is level with BM25 (+0.002, interval [-0.005, +0.010]). With nine units these intervals are wide, and the check says nothing about exact code facts in these languages.

6.6 Third held-out test, version 0.5.0

What changed. Against version 0.4.1 the retrieval path has four changes. The relevance models were refitted on all 34 RepoMem repositories, keyed and unkeyed, and on the conversation sets, with chats and code as two families of equal weight. A newer near-twin now replaces an older note only if it covers the query at least as well. Twins compared by hashed words have their own removal threshold of 0.85. And the plugins have a query gate, measured on its own in Section 6.10. All of this was tuned on repositories the first two tests had used, so those tests cannot judge it. The static model2vec model was not refitted and is not part of this test.

The training recipe was picked by leave-repository-out recall. Scored out of fold over all budgets, the new recipe moves mean recall from 0.457 to 0.494 on the TypeScript, Go and Rust repositories, from 0.861 to 0.870 on the second test's repositories without keys, and from 0.799 to 0.803 on LongMemEval-S. It costs a little on LoCoMo, from 0.733 to 0.726. The lexical model moves from 0.434 to 0.461 on the same three languages and stays at 0.660 on LoCoMo.

The test. Fifteen repositories that no fit, threshold or design choice had used, with 2,148 queries over three seeded streams each: scrapy, celery, httpie, sanic, boto3 and channels in Python, vue core, TanStack query and immer in TypeScript, cobra, echo and bubbletea in Go, bat, hyper and fd in Rust. The method was frozen first and the test ran once, so nothing here was tuned on its results. Code review afterwards fixed defects in the plugins and in one experiment script, and a rerun of sampled rows matched the stored results exactly. The fact track reads Python source, so it exists only for the six Python repositories, and the other nine have task queries alone. With fifteen units, significance is reachable. `kanerva-0.4` and `kanerva-lex-0.4` are the released 0.4.1 method, with its model and rules. Tables 10 and 11 give the results, and Figure 1 shows recall by budget.

Recall. Without keys, averaged over budgets, KANERVA has 0.775 against 0.752 for hybrid and 0.749 for the reranker. That is +0.024 over hybrid, unit interval [+0.014, +0.033], with 15/15 repositories in its favour at p <0.001, and +0.026 over the reranker, interval [+0.018, +0.037]. It leads at every budget. With keys the difference to hybrid is +0.025, interval [+0.016, +0.034], and to hybrid with invalidation +0.017, interval [+0.010, +0.023]. Unlike in the second test, the lead no longer depends on keys. Keys can matter only on the six repositories with facts, and there the lead is much the same with them and without them.

Against the released 0.4.1 the gain is +0.022 without keys, interval [+0.017, +0.026], and +0.010 with keys, interval [+0.006, +0.015]. The query condition on near-twins accounts for +0.013 of the first figure, interval [+0.006, +0.018]. It moves nothing with keys, where twins are not used, and it can act only on the six repositories with facts, which is why it shows 6/15. Without keys at 1024 tokens, task recall is 0.569 against 0.527 for hybrid. Fact recall is 0.993 against 0.998, so the method still gives up a little on exact facts.

Lexical. This is what a store runs with `KANERVA_EMBEDDER=off` or until the model has arrived. It stays close to BM25. Averaged over budgets without keys it has 0.739 against 0.727, a difference of +0.013 with interval [+0.002, +0.022] and 9/15 repositories in its favour. Nearly all of that comes from the smallest budgets. From 512 tokens up the two are close to level: +0.003 at 512, +0.002 at 1024 and +0.003 at 4096, with intervals [-0.009, +0.019], [-0.005, +0.009] and [-0.002, +0.009]. Against its own 0.4.1 it is level on average (+0.003, interval [-0.000, +0.005]), and behind at 1024 tokens (-0.005, interval [-0.007, -0.003], 3/15 repositories in its favour). That loss is the price of the lower removal threshold: fact recall at 1024 tokens is 0.984 against 0.992, in exchange for the drop in stale injection below.

Languages. Table 12 splits the result. Averaged over budgets, KANERVA leads hybrid in each language: 0.870 against 0.844 on Python, 0.545 against 0.529 on TypeScript, 0.523 against 0.509 on Go and 0.441 against 0.425 on Rust. Each of the last three rests on three repositories and the task track alone. The lead does not hold at the smallest budgets there: at 256 tokens it has 0.291 against 0.294 on Go, and on Rust 0.158 against 0.159 at 128 tokens and 0.240 against 0.242 at 256. The earlier check on other TypeScript, Go and Rust repositories found no lead with a model that had seen code only from Python (Section 6.5), so adding such repositories to the fit is the likely cause of the change. The lexical model only ties BM25 outside Python, at 0.521 against 0.520 on TypeScript, 0.470 against 0.467 on Go and 0.396 against 0.393 on Rust.

Staleness. Table 13 covers queries whose fact changed, at 1024 tokens, which exist only on the Python repositories. With keys every method that follows version chains injects no stale version. Without keys, hybrid and the reranker inject one 99.9% of the time. KANERVA does so 34.9% of the time and the lexical model 26.6%. The plain rule of keeping the newest near-twin reaches 36.5%, so here too the gain comes from that rule and not from the validity model. The query condition raises it slightly, from 33.9% to 34.9%. The error, a stale version listed above or instead of the current one, stays near zero at 0.003 for KANERVA, against 0.722 for hybrid.

Flaws in the facts. Two flaws came out of the answer test. Of the 1,772 changed facts, 287 (16.2%) only moved to another file and kept their value, so injecting the older note does no harm, yet the stale rates count them. Without them, at 1024 tokens and without keys, stale injection is 36.1% for KANERVA instead of 34.9%, 31.2% lexically instead of 26.6%, and 38.0% for the newest-twin rule, so the lexical figure above flatters the method a little. The other flaw is small: 7 dependency queries read console-script entries as version constraints. Without all of these queries the lead over hybrid moves from +0.024 to +0.024, with 15/15 repositories in its favour. The dataset stays as it was, so that the tests remain comparable.

A correction. The project README gave a stale rate of 24.3% without keys next to the lexical plugin default. That figure held only with MiniLM vectors. Benchmark stores held vectors even in lexical runs, so the old lexical rows compared near-twins by vectors, which a real lexical store does not have. On the third test, the real lexical figure for version 0.4.1 was 95.5%. The benchmark now compares twins by hashed words in lexical runs, and the lexical rows of this section are measured that way. The lexical rows of the first two tests were not rerun.

6.7 C and C++, version 0.5.0

The tests above hold no C or C++, a large share of what coding agents work on. We added ten repositories that nothing had used, with facts read by regular expressions in place of Python's `ast`: literal macros and constants, public function signatures, and where each function and type is defined. Four of them, tmux, libuv, nlohmann/json and googletest, served to find what these languages break. They broke nothing in the retrieval path, so the method went to the test unchanged. The plugins gained CMake, ctest, `make check` and Ninja runs as test commands, `Class::method` as a name, and common C and C++ file names. Pairing

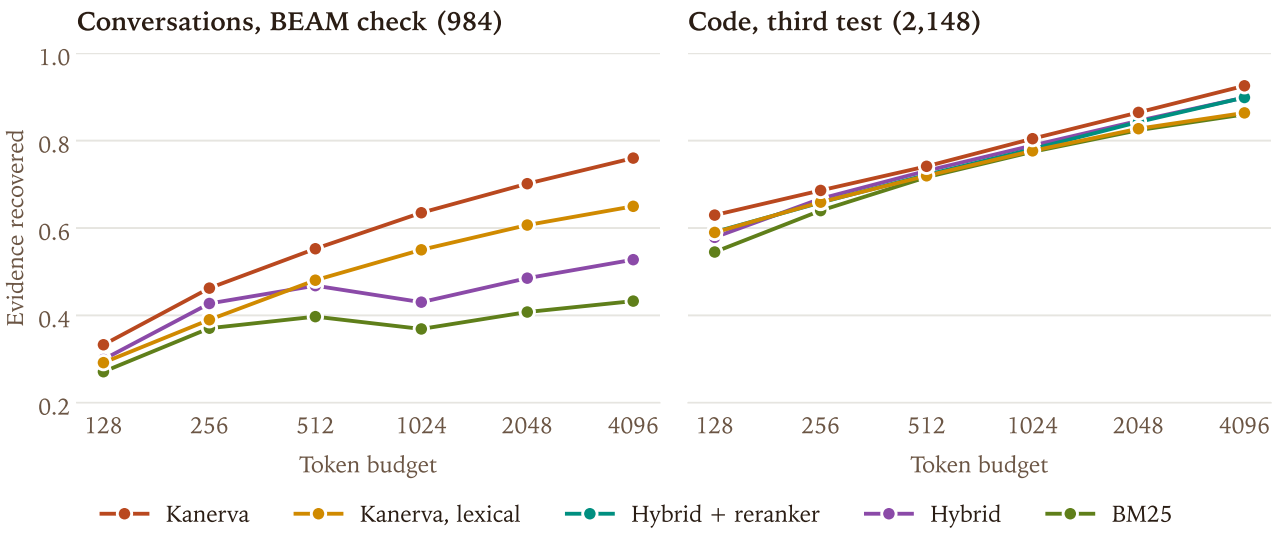


Figure 1: Evidence recall at each token budget. Left: the BEAM check, on data that earlier tests saw. Right: the third held-out test, without keys. The reranker was not rerun on BEAM.

a header with its source file in file notes lost a little on these four and was dropped.

The test is the other six: redis, curl and jq in C, fmt, spdlog and leveldb in C++, with 1,534 queries over three seeded streams each. It ran once. The relevance model has seen neither language, and with six repositories the smallest possible p is 0.031. Table 14 gives recall without keys.

KANERVA leads hybrid at every budget, with 0.873 against 0.859 averaged over budgets, a difference of +0.014 with interval [+0.010, +0.017] and 6/6 repositories in its favour. With keys it is +0.017. The lead over the reranker is smaller, +0.006 with interval [+0.001, +0.011], and at 128 tokens it is behind at -0.003. Against version 0.4.1 the gain is +0.012. Averaged over budgets it has 0.867 against 0.855 on C and 0.879 against 0.863 on C++. The gain is on tasks, 0.538 against 0.482 at 1024 tokens. Facts sit near the ceiling, because a C identifier is rare enough to find by itself, and there KANERVA gives up a little, at 0.994 against 0.999.

The lexical model only ties BM25, with +0.001 over budgets, and it is behind from 512 tokens up, at -0.006 and -0.007. It is also behind its own 0.4.1, at -0.010, and more so on C++, with 0.834 against 0.845 for BM25. So a C++ project should let the model arrive.

On changed facts at 1024 tokens without keys, hybrid injects a stale version 99.1% of the time, KANERVA 51.4%, the newest-twin rule 53.7% and the lexical model 23.5%, against 0.0% with keys. The refit made KANERVA a little worse here than 0.4.1 at 49.9%. As of version 0.5.2, file notes gain +0.099 at equal tokens when the file belongs to the task, better on 6/6 repositories, and lose -0.004 when it belongs to the last one. The gate takes foreign same-kind questions from 0.937 to 0.253 at a recall of 0.818 against 0.847.

The extractor is cruder than `ast`: templates and overloaded names yield no fact, and a macro counts only if its value is a literal. Section 6.18 describes live sessions in a CMake project.

6.8 A second C and C++ test, version 0.5.1

That test left weak spots: 51.4% stale injection without keys, a lexical model that only tied BM25, and a trail behind the reranker at 128 tokens. Refitting was not the answer. Out of fold, adding the C and C++ repositories to the fit moved the test's mean recall by +0.002 with vectors and +0.005 lexically, and LongMemEval-S by -0.005, so the models carry over as they are and we kept them.

The cause was the near-twin rule. A sentence model hears little difference between `FOO = 3` and `FOO = 4`, so twins by vectors missed most changed constants. Twins by hashed words catch them, but they also fired on neighbouring names, which cost the lexical model recall. Version 0.5.1 compares twins both ways. A newer twin must now hold the query's words whole, since `dictAddRaw` says nothing about `dictAdd`, and a note that says less by more than one word is not a newer version. All of this was set on repositories already spent. We then froze the method and ran it once on six repositories nothing had used: nginx, libgit2 and libevent in C, Catch2, benchmark and imgui in C++, with 1,537 queries. Table 15 gives recall without keys.

KANERVA leads hybrid by +0.011 over budgets, interval [+0.006, +0.016], 6/6 repositories in its favour, and

the reranker by +0.012, interval [+0.007, +0.018], 6/6, now ahead at 128 tokens too, at +0.009. No per-budget test can pass correction with six repositories, and the interval over hybrid at 128 tokens includes zero. It has 0.918 against 0.902 on C and 0.837 against 0.831 on C++, and on tasks at 1024 tokens 0.553 against 0.477. The lexical model now leads BM25 by +0.009, interval [+0.003, +0.014], 5/6 repositories.

Stale injection without keys at 1024 tokens is 16.0%, against 42.9% for the rule of 0.5.0, 96.9% for hybrid and 44.2% for the newest-twin rule. Here the method does clearly better than that rule, which it did not on the earlier tests. The lexical model has 25.5%, a little worse than 23.4% under the old rule, and with keys all are at 0.0%. A real agent on these repositories fixed 12 of 12 changed facts, and it left the store alone on 10 of 12 other notes. The two others retired a location note that the new note repeats.

There are costs. Strict fact recall at 1024 tokens is 0.992 against 0.996 under the old rule and 0.998 for hybrid, so the rule costs -0.003 over budgets. Most of that is a label effect: the value itself reaches the context on 0.9995 of facts, against 0.9995 and 0.9996, because a newer note holds it. On C++ facts it trails hybrid over budgets, 0.980 against 0.990. The gate takes foreign same-kind questions from 0.983 to 0.227 at a recall of 0.836 against 0.862.

The change reaches every language, so we checked it on data that has been read, which is a check and not a test. On the third test's repositories recall moves by -0.001 with vectors, and stale injection falls from 34.9% to 21.0%. The lexical model moves by +0.002, and its stale injection rises from 26.6% to 29.0%. On BEAM nothing moves, at +0.000. Apart from the write-path runs, the other results in this paper describe version 0.5.0, the answer test included.

6.9 A third C and C++ test, version 0.5.2

Version 0.5.2 changes file notes. Ranking them by the prompt raised their precision but not their worth, because it brought what the prompt had already found, and dropping notes that share no word with the prompt lost most of the gain. Two other changes worked on spent repositories. The hook does not repeat what the prompt brought, and a file that at least five percent of a store's notes name gets no notes, since it says nothing about any of them. The numbers in Section 6.12 now include both. We also tried to win back the costs above. Letting a note that says more stand beside an older one raised strict fact recall but brought stale injection back, and relaxing the rule for lexical stores cost long notes, so the near-twin rule stayed.

This test also reads what the earlier ones could not: templates, overloads listed together, and methods under their class. We froze the method and ran it once on six new repositories, lz4, zstd and expat in C, cpp-httplib, CLI11 and yaml-cpp in C++, with 1,461 queries. Table 16 gives recall without keys.

File notes did what they were changed for: +0.077 at equal tokens when the file belongs to the task, better on 5/6 repositories and worse on 0/6, and -0.012 when it belongs to the last task.

Retrieval is mixed, and the lead over hybrid does not hold here. Averaged over budgets without keys it is +0.005, interval [-0.014, +0.023], 4/6 repositories, and against the reranker -0.000, interval [-0.018, +0.017]. On C it leads, with 0.939 against 0.914. On C++ it trails, with 0.816 against 0.827 for hybrid and 0.832 for the reranker, although it leads on tasks, 0.414 against 0.351. The loss is on facts, 0.947 against 0.983. Of 3,564 fact queries, 84 lose a note at 1024 tokens that hybrid keeps, and 74 of those are location notes, most of which gave way to a newer signature note naming the same file. The value still reached the context on 0.992 of the lost queries, and overall on 0.9992 against 0.9994 for hybrid, so most of this is the label. With keys, where the rule does not apply, the lead over hybrid is +0.023, on 6/6 repositories, and over the reranker +0.017.

The lexical model is level with BM25 at +0.001, and behind from 512 tokens up, at -0.014. Stale injection is 16.4% against 44.7% under the rule of 0.5.0 and 98.0% for hybrid, 20.5% for the lexical model, and 0.0% with keys. The gate takes foreign same-kind questions from 0.977 to 0.187 at a recall of 0.823 against 0.854. The facts of this test differ from those of the two before, so its numbers do not compare with theirs directly.

6.10 Query gate

Without a gate a store with vectors speaks on nearly every prompt. To measure this, each store answers its own questions, questions from other stores of the same kind, and questions from another kind of data, at the plugin budget of 600 tokens. Table 17 gives the results. The third-test row uses the shipped gates on twelve sampled streams they never saw. The other rows are out of fold: a set is scored by a gate fitted on the others.

On the third test with vectors, the share of foreign same-kind questions that get any notes falls from 1.000 to 0.203, and of other-kind questions from 1.000 to 0.000. Lexical stores fall from 0.963 to 0.413 and from 0.617 to 0.017. So a lexical store still speaks on two in five questions from another repository. On conversations the lexical gate does little: from another conversation, LongMemEval-S still speaks on 0.900 of questions and BEAM-1M on 0.887.

This has a cost on a store's own questions. Their recall falls from 0.657 to 0.620 with vectors, a relative loss of 5.6%, and from 0.591 to 0.573 lexically, a loss of 3.0%. With vectors that is more than the 3.0% the

threshold allows on the data it was fitted on. With the gate, 0.897 of a store's own questions still get notes with vectors and 0.942 lexically, so the rest get none. Out of fold the cost is largest on the earlier TypeScript, Go and Rust repositories, from 0.531 to 0.446, a loss of 16.0%, and on LongMemEval-S, from 0.979 to 0.896 on only 12 questions.

The false-alarm numbers come from foreign questions. A template question from another repository names identifiers this store has never seen, which makes it easier to reject than a vague prompt about the store's own project. Section 6.11 shows how much this matters. `KANERVA_GATE=0` turns the gate off.

The gates were fitted on MiniLM scores. On five sets with the static embedder, averaged over the sets, they keep own recall at 0.735 against 0.768 without a gate and 0.734 for a gate fitted out of fold on static scores, and foreign same-kind questions get notes on 0.445 of cases against 0.426. So the shipped gate serves that embedder too.

6.11 Real prompts

Every prompt above comes from a template or a dataset. Here we replay one developer's own Claude Code transcripts through the hooks, project by project and in time order, into throwaway stores: 603 prompts in 29 sessions over 5 projects, 299 of them in Russian. Prompts from another project stand in for prompts a store cannot answer. A store holds what the hooks write, and where the wrap-up rule asks for a note, the turn's last reply stands in for it. Prompts stayed on the machine and the results hold counts only. This is a small sample of one person's work, and anyone can run the script on their own transcripts and get other numbers.

The gate separates much less here than on the benchmarks. With vectors, own prompts get notes 69.8% of the time without it and 53.6% with it, and prompts from other projects 51.3% and 27.7%. Lexical stores go from 49.9% to 45.9% on own prompts and from 37.3% to 34.0% on foreign ones, so there it barely separates them. Language matters: with vectors, English own prompts get notes 69.4% of the time with the gate, and Russian ones 37.5%, because these notes are mostly in English. The gate still halves the noise, at a real cost in silence, and its false-alarm rates in Table 17 do not carry over to real use.

A weak label says when a store could have helped: one of its notes names a file that this turn then touched, which holds for 174 prompts. On those the gate speaks 65.5% of the time, against 77.0% without it, but the notes that a prompt brings name that file only 13.2% of the time. So a real prompt rarely finds the note about the file the agent will open, which is the case for file notes.

These sessions never had these notes, so an agent cannot have used them, and each time the use signal of Section 6.16 fires here it is wrong. It fires on 20.5% of the notes a prompt brought, and on 13.0% of random notes from the same store. A signal that fires this often on notes nobody saw cannot tell use from coincidence, so learning stays opt-in. The same replay showed 17.6% of the lines at session start to be lists of edited files, which only repeat the git log, so the plugins now leave them out.

6.12 File notes

RepoMem's task track gives each query the files of a later commit, so it can simulate two sessions at the two extremes. These results use version 0.5.2, where a file that many notes name gets no notes and nothing the prompt brought is repeated. In the first the agent has just opened one file of the change. In the second it has just worked on up to three files of the commit before, so its files belong to the previous task. We compare at equal tokens: the prompt at 512 tokens plus file notes, against the prompt with those tokens added to its budget. Table 18 gives the results with vectors.

On the third test with the file open, recall rises from 0.430 to 0.560, against 0.463 at equal tokens, a gain of +0.096 and better on 15/15 repositories, for 80 tokens of notes. When the files belong to the previous task, recall is 0.445 against 0.478, a loss of -0.032, worse on 15/15. The notes' precision shows why, 0.289 against 0.024. Lexical stores behave alike, with +0.111 and -0.026.

These labels are weak. Task relevance comes from file overlap, so a note that names a file of the change is close to correct by construction, and the first session is the generous case. A real session sits somewhere between the two, and we do not know where.

Adding the session's files to the prompt query was tested and dropped. Lexically it reaches 0.594 when the file belongs to the task, but 0.338 when it belongs to the last one, below the prompt alone at 0.399, so it hurts exactly when the session has moved on. This was measured lexically only, because the embedding cache holds no vectors for expanded queries. File notes cost at most their budget, and they do not displace what the prompt found.

Delivery differs by host. On Claude Code, checked live with a real model, the note reaches the model from the step after the tool call, because the hook is asynchronous. Copilot CLI delivered it in the very next request, and Codex delivered a hook's context one step later, both with a stub model. A stub model is not offered Codex's patch tool, so the trigger itself was not run there. OpenCode and Hermes hold the notes for the next request, which only unit tests cover.

6.13 The write path without keys

A note without a key replaces an older one only if someone says so, so `memory_remember` lists similar facts and the host agent decides (Section 4). We replay streams with no keys and ask for this list as each note is written. No LLM takes part. Conversation turns are replayed as facts, since an agent's notes are facts. The word thresholds were picked on development streams, the cosine threshold comes from the real agent below, and Table 19 reports streams they were not picked on.

On the third test with vectors, the older version of a changed fact comes first on 0.931 of changed notes, and it is on the list on 0.994. Lexically the figures are 0.898 and 0.952. Natural language is much harder: on LongMemEval updates they fall to 0.511 and 0.809, on only 47 changed notes.

The list has many false alarms. On the third test it appears on 100.0% of the notes that replace nothing, and across all lists the top hit is something other than an older version 80.6% of the time. On LongMemEval the rates are 92.6% and 99.8%. Each list costs about 154 tokens of the agent's context. So an agent that replaced whatever came first would retire good notes, and the value of the list depends on the agent's judgement.

The stale columns are a simulated upper bound. An oracle stands in for the host agent and marks the true older version whenever the list shows it. Stale injection on the third test's changed facts then falls from 21.0% to 0.3% with vectors, and from 29.0% to 3.5% lexically. On LongMemEval updates it falls from 100.0% to 31.9%. These numbers say what a perfect agent could reach.

A real agent. Here Claude Code with the plugin and Sonnet 5.5 decides for itself. Each trial gets a store replayed up to a note and an instruction to remember it, and the store afterwards shows what the agent did. A *verbatim* note keeps a text word for word. An *own* note is what an agent normally writes: a first session turns the older conversation turn into a short note in its own words, and a second, fresh session does the same with the newer turn.

The first round failed on conversations. On the third test's changed facts the agent retired the older version 9 times out of 16, and on LongMemEval updates 0 times out of 8. Its replies gave the reasons, round by round. A conversation turn holds several subjects, so it cannot be retired whole, and the hint now says to take the outdated part out with `memory_edit`. An instruction inside a tool result did not count as a request from the user, so the server's instructions now say that keeping memory current is part of saving. Paraphrases of one fact missed a cosine threshold of 0.95 that came from word-for-word data, so it is now 0.3, in a list of five that alternates word matches and close vectors. A list showed the opening of a long note, while the outdated detail sat further on, so it now shows the sentence closest to the new note. A personal note saved as global saw only global notes, so a note now sees both. And the agent left an old value alone as a dated snapshot, so an edit now keeps the earlier text as history, and the instructions say that notes state what is true now while a dated event stays. We also dropped six LongMemEval questions that ask about the past, since there the older fact is the answer. In the last sample two notes on one fact sit from 0.31 to 0.83, median 0.61, and notes on different subjects reach at most 0.20.

On that last sample of 80 trials, with the code as shipped, the agent fixed 10 of 12 changed facts in its own conversation notes (interval 55–95%), nearly all by correcting the older note, 10 of 12 in word-for-word conversation turns (55–95%), and 16 of 16 on code facts (81–100%). We read the four conversation misses. In three the agent gave a reason one can defend: an earnings list whose dated entries stay true while its total does not, a wake-up time it asked to confirm, and a finished model it took as a past project. In the fourth the older note was not on the list. Pushing harder on these would mean editing notes that are still true.

On notes that replace nothing it left the store alone in 38 of 40 trials. Over the whole sample, in 7 trials it changed a note the labels did not mark, and we read them all. Most were right or harmless: a rename the labels miss, a second copy of the old value, or a note the new one repeats whole. One was a real loss: with a moved function the agent also retired its signature note, which now survives only as history.

Word-for-word trials were drawn from notes whose older version the list showed, so they say what the agent does with the list and not how often it helps. Own notes had no such filter. These are small samples of one agent and one model, every fix was made after reading earlier rounds, and each round used new notes, so only the last sample is unseen, and its intervals are wide. Results moved by a few trials between rounds. Without vectors the list stays word-based and misses most paraphrases.

6.14 BEAM check

BEAM-100K and BEAM-500K were never fitted on, but earlier tests saw them, so this is a check and not a test. No untouched conversation benchmark is left.

Averaged over budgets, Kanerva has 0.574 against 0.440 for hybrid, a difference of +0.134 with unit interval [+0.124, +0.147] and 55/55 conversations in its favour. At 1024 tokens it has 0.635 against 0.430. The lexical model has 0.495 against 0.375 for BM25. The refit moved little here: +0.005 against version 0.4.1, interval [+0.000, +0.009], with small losses at 2048 and 4096 tokens (-0.003 and -0.002) that are inside their intervals. Scored out of fold, LoCoMo went from 0.733 to 0.726 with the refit, which is the cost of giving code

the same weight as chats. The reranker was not rerun; its figures from the first test are in Table 4.

6.15 Russian and cross-language search

Russian support adds a Snowball stemmer, stopwords and query patterns, and `KANERVA_EMBEDDER=multilingual` adds search across languages. They act on Cyrillic text or when asked for, so English behaviour is unchanged. We test on XQuAD [5], which gives SQuAD paragraphs and questions in English and in professional Russian translation: 48 articles, each a store of five paragraphs, with 1,190 questions, and one pooled store of all paragraphs. A retrieval counts if it returns the question's paragraph, and the unit is the article. This is not coding-agent memory, and the relevance model has not seen Russian.

Stemming helps once a note fits. With article stores it moves lexical recall from 0.897 to 0.954 at 1024 tokens, a difference of +0.056 with interval [+0.040, +0.073], and in the pooled store from 0.861 to 0.933. At 128 and 256 tokens it barely moves, at 0.021 and 0.098, because a Russian paragraph averages 400 tokens against 163 in English, and a whole note rarely fits. With excerpts, which are off by default, recall at 128 tokens is 0.727.

For a Russian question over English notes, lexical search finds almost nothing, 0.152 in the pooled store at 1024 tokens. MiniLM does well inside a store of five paragraphs, at 0.966, but fails in the pooled store, at 0.182. The multilingual model reaches 0.985 and 0.810. On English it gains nothing: in samples of LoCoMo and of the second test's repositories at 1024 tokens, the difference to MiniLM is -0.007, interval [-0.018, +0.000], and -0.006, interval [-0.013, +0.002]. It also costs more, 127 MB against 91 MB on disk and 0.56 s against 0.12 s to load. So MiniLM stays the default.

6.16 Online learning

Feedback here is simulated from evidence labels, as described in Section 7, and these runs use the current models. Each seed shuffles the order of work and shares a single policy. Table 20 reports recall at $B = 512$. LongMemEval-S gives each question its own store, so nothing carries over, and every column is the same. The linear bandit loses to the fitted model: on LoCoMo recall moves from 0.711 to 0.668, and on RepoMem from 0.902 to 0.885. Past use of an item for another query turns out to be a poor predictor for the next one.

Item lift, the plugins' opt-in path, asks a narrower question, whether this note helps more often than the model expected. On simulated feedback it gains a little on LoCoMo, 0.717 against 0.711, and it is flat on RepoMem, 0.902 against 0.902. With 20% of rewards flipped the small gain goes, at 0.710 on LoCoMo and 0.901 on RepoMem. So it does no harm in these runs and adds very little. The signal it would use in a plugin, identifiers of a note reappearing in later tool calls, has not been validated on real sessions, so learning stays opt-in.

6.17 Answers from a local model

Recall says what reaches the context and not what a model does with it, so here a small local model answers each question from the notes a method selected. The memory layer still calls no LLM. The reader is Qwen3-4B in 4-bit form on MLX, pinned by revision, and the prompt tells it to answer from the notes alone, in as few words as it can, or to say unknown. We take 60 questions from LoCoMo, 60 from LongMemEval-S, 40 from BEAM-1M and 90 fact queries from the six Python repositories of the third test, without keys, at 256, 512 and 1024 tokens. The first three are development data. The fourth is the only set here that no decision used, and with six repositories the smallest possible two-sided p is 0.031.

Only short extractive answers can be scored without a judge. LoCoMo keeps categories 1 and 4, LongMemEval-S drops temporal reasoning and preferences, and BEAM keeps knowledge updates. Two categories went after a pilot because the reader missed them even when handed the labelled notes, and no method comparison informed that. Scoring rules were fixed on small pilot runs before the full run. The score counts an answer as right if it has every word of the reference, in any order, or for code the whole value as an unbroken run. That is strict: a reply of "four months" fails against "about four months". With no notes the score is zero on every set. The last row of Table 21 hands the reader the labelled evidence alone. It is a reference and not a bound, and on BEAM-1M some methods pass it.

Methods. Across every comparison and metric, 7 of 264 tests reach raw $p < 0.05$ and 0 survive Holm correction, so this test shows no reliable difference between methods. Of the 20 means in Table 22, 1 reaches raw $p < 0.05$: on the third test's facts Kanerva beats BM25 by +8.5 points, interval +4.5 to +14.4, on all six repositories. One such result in that many comparisons is about what chance gives. The intervals are wide, so a null here does not mean two methods are equal.

Recall against answers. On conversations recall barely converts. At 512 tokens Kanerva leads BM25 in recall by +13.0, +21.5 and +20.5 points on LoCoMo, LongMemEval-S and BEAM-1M, and its answers move by -1.7, +0.0 and -5.0 points. On the third test's facts the recall difference at 512 tokens is +1.1 points, and answers move by +11.1. We did not test why.

Losses. Many differences go against Kanerva. Against BM25 they are -10.0 and -5.0 points on BEAM-

1M at 256 and 512 tokens, -5.0 on LongMemEval-S at 256, and -1.7 and -1.7 on LoCoMo at 256 and 512. Against hybrid they are -7.5 and -10.0 on BEAM-1M at the same two budgets. The lexical model is behind BM25 on each conversation set on average, and on the third test at 256 tokens, at -2.2. Against version 0.4.1 the mean moves by +0.0, -1.7 and -5.0 points on the three conversation sets, so the refit shows no gain there, and by +4.4 on the third test. None of these losses survives correction, and neither do the gains.

Stale answers. Table 23 covers the 35 sampled facts on the third test whose value changed. Hybrid and BM25 put a stale note in the context nearly every time, yet the answer gives the old value far less often, so retrieval stale rates overstate the harm for this reader. KANERVA gives the old value in 0.0%, 0.0% and 5.7% of answers against 8.6%, 11.4% and 14.3% for hybrid. At 1024 tokens it is worse than 0.4.1, at 5.7% against 2.9%. The largest change is lexical, from 17.1% to 2.9% at 512 tokens. Over budgets that is -17.1 points at p 0.062, and against BM25 KANERVA has -14.3 points at p 0.062.

Foreign questions and the gate. On 160 questions from other sources at 512 tokens, which no store can answer, KANERVA, its lexical model, hybrid and BM25 invent an answer 4, 5, 4 and 4 times, and with no notes 0 times. The gate lowers the share of these questions that get any notes, on average to 28.1% with vectors and 64.4% lexically, but invented answers stay at 4 and 4. With this reader the gate saves tokens and not mistakes, because the reader already refuses. On a store's own questions at 512 tokens it moves answers by -3.3 points on the third test and -1.7 on LongMemEval-S.

6.18 Live sessions

We also ran the plugin in real Claude Code sessions with Sonnet 5.5, on small Python and C++ projects with an isolated store. Memory carried over: after one session found that the machine has no `python`, the next went straight to `python3`, and after a C++ session learned that the tests need a Debug build for their assertions, the next ran that CMake command first. Asked to rebuild as last time, a later session ran the exact Release and Ninja command from a stored build note. No secret reached the store when a session was told to remember a token, although the agent wrote it to the host's own memory files.

These sessions found defects no benchmark could. Notes held absolute paths. Edits to the host's own memory files counted as work and became notes. A test command that was never found was kept as the project's test command. File notes repeated what the prompt had just brought. Requests to remember something went to the host's own memory and not to the store, until the server's instructions said otherwise, after which three of three went to the store. And in `claude -p` the wrap-up turn replaced the message a script reads, so it is now off in that mode. All of these are fixed. These runs are anecdotes, not measurements.

6.19 Excerpts

Turn recall credits an excerpt that lost the answer, so excerpts are compared on *answer-term recall*: the share of the reference answer's content stems, excluding those in the question, found in the injected text. In the BEAM run of the first test, at 128, 256 and 512 tokens, excerpts of long items raise it from 0.280, 0.384 and 0.475 to 0.289, 0.395 and 0.489. The differences are small, +0.010 to +0.013, and they fade by 1024 tokens, so the mode stays off by default.

6.20 Checks of the theory on synthetic instances

With known ground truth (eight seeds, horizon 4000, 40 items per round, three classes with different hazards), cumulative regret against the per-round greedy optimum is 22 for a learner that knows validity, 26 when it estimates hazards from censored lifetimes, and 2015 when it is blind. The log–log slopes in T are 0.26, 0.25 and 0.97. The blind learner's linear growth is what Proposition 9 predicts, and it shows the value of the separation. The observed regret of the other two is far below the upper bound of Theorem 2, whose value here exceeds the trivial bound, so these runs do not test the bound's tightness. Over 1600 random selection problems, $F(S)$ was on average 0.707 of the certificate upper bound, with a minimum of 0.600, above the worst-case constant of 0.316.

6.21 Consolidation and forgetting

These results come from the earlier version and were not rerun, since both steps stay opt-in.

Consolidation. Table 24 applies consolidation at several thresholds. A gold item counts as found if it or its summary is selected, which measures discoverability, since the sources stay expandable. Consolidation is not free. At $\tau = 0.9$ it merges only near-duplicates: it saves 2.1% of tokens on the LongMemEval-S sample and 0.5% on LoCoMo, with recall unchanged. At $\tau = 0.8$ it saves 19.9% and 5.6%, and recall moves from 0.969 to 0.789 and from 0.824 to 0.791. At $\tau = 0.7$ the loss is larger. Since the budget caps injected tokens anyway, a smaller store buys little here, so the default threshold is 0.9.

Forgetting. On RepoMem streams, Table 25 compares keeping everything with idle-time rules and token capacities. Forgetting has a real cost on this benchmark. Idle rules of 365 and 180 days cut the active store to 6176 and 3227 tokens from 21492, as Theorem 4 leads one to expect, but recall falls from 0.923 to 0.713 and 0.551, because long-lived facts are read rarely and are still correct. Sessions in this benchmark are 168 days apart, so these idle rules are short in session terms, and the numbers do not transfer directly to a daily workflow. Token capacities behave similarly. Given the latencies below, a store of this size does not need forgetting, so the rules are opt-in. Archived items remain recoverable by hand, but they leave retrieval.

6.22 Latency and scale

On an M1 Pro with an uncontended CPU, a fresh query at 10^4 items takes 287.1 ms at the median and 359.7 ms at the 95th percentile, plus 8 ms to embed the query, and lexical retrieval takes 150.2 and 210.5 ms. At 10^5 items the medians are 332.4 and 182.2 ms. A write costs 0.49 ms, excluding the embedding. These timings come from the earlier version. In a plugin, a hook starts a new process each time, so one prompt at 10^4 items takes 0.72 s in all with MiniLM, 0.49 s lexical and 0.98 s with the static embedder. The memory layer spends no LLM tokens, so its token cost is the injected context alone.

Table 4: First held-out test, earlier version: evidence recall. Bold marks the best method per column within a dataset.

Method	128	256	512	1024	2048	4096	mean
BEAM-100K and 500K							
KANERVA + reranker	**0.350**	**0.480**	**0.571**	**0.644**	**0.707**	0.753	**0.584**
KANERVA	0.324	0.449	0.537	0.635	0.699	**0.754**	0.566
KANERVA, lexical	0.281	0.388	0.474	0.546	0.599	0.641	0.488
First version	0.312	0.434	0.494	0.493	0.549	0.600	0.480
Hybrid + reranker	0.322	0.439	0.512	0.488	0.533	0.576	0.478
Hybrid RRF	0.300	0.428	0.468	0.430	0.484	0.529	0.440
Hybrid, live only	0.300	0.428	0.468	0.430	0.484	0.529	0.440
Hybrid, newest of twins	0.300	0.428	0.467	0.430	0.485	0.528	0.440
Dense	0.272	0.380	0.458	0.450	0.506	0.553	0.436
BM25	0.271	0.371	0.397	0.368	0.408	0.431	0.374
MMR	0.257	0.356	0.446	0.442	0.504	0.562	0.428
Generative Agents	0.012	0.019	0.024	0.021	0.024	0.028	0.021
Facility location	0.065	0.125	0.222	0.369	0.538	0.668	0.331
Recency	0.006	0.008	0.010	0.008	0.012	0.013	0.009
RepoMem, keys							
KANERVA + reranker	**0.782**	**0.809**	**0.835**	0.866	0.895	0.926	0.852
KANERVA	0.782	0.805	**0.835**	0.869	0.896	0.926	0.852
KANERVA, lexical	0.721	0.780	0.817	0.847	0.884	0.904	0.826
First version	0.759	0.804	0.827	0.857	0.883	0.915	0.841
Hybrid + reranker	0.726	0.796	0.831	0.871	0.916	0.951	0.849
Hybrid RRF	0.712	0.799	0.822	0.869	0.915	0.951	0.845
Hybrid, live only	0.764	0.802	0.833	**0.880**	**0.919**	**0.952**	**0.858**
Hybrid, newest of twins	0.756	0.799	0.830	0.874	0.908	0.927	0.849
Dense	0.734	0.798	0.830	0.868	0.906	0.940	0.846
BM25	0.650	0.763	0.813	0.850	0.893	0.939	0.818
MMR	0.687	0.784	0.821	0.866	0.909	0.945	0.835
Generative Agents	0.211	0.381	0.659	0.825	0.900	0.940	0.653
Facility location	0.103	0.168	0.251	0.378	0.549	0.791	0.374
Recency	0.024	0.048	0.077	0.116	0.210	0.370	0.141
RepoMem, no keys							
KANERVA + reranker	0.744	0.786	0.819	0.852	0.883	0.916	0.833
KANERVA	0.750	0.786	0.821	0.855	0.884	0.917	0.836
KANERVA, lexical	0.698	0.758	0.800	0.833	0.872	0.895	0.809
First version	0.713	0.776	0.809	0.845	0.871	0.906	0.820
Hybrid + reranker	0.726	0.796	**0.831**	**0.871**	**0.916**	**0.951**	**0.849**
Hybrid RRF	0.713	**0.799**	0.822	0.869	0.915	**0.951**	0.845
Hybrid, live only	0.713	**0.799**	0.822	0.869	0.915	**0.951**	0.845
Hybrid, newest of twins	**0.754**	0.798	0.826	0.870	0.904	0.923	0.846
Dense	0.734	0.798	0.830	0.868	0.906	0.940	0.846
BM25	0.650	0.763	0.813	0.850	0.895	0.939	0.818
MMR	0.688	0.785	0.821	0.866	0.909	0.945	0.836
Generative Agents	0.211	0.380	0.657	0.825	0.900	0.940	0.652
Facility location	0.103	0.168	0.252	0.380	0.550	0.792	0.374
Recency	0.024	0.048	0.076	0.113	0.209	0.371	0.140

Table 5: First held-out test, earlier version: paired differences in recall. Positive favours the first method named. Unit CI resamples conversations or repositories, Units + counts those favouring the first method, and p is Holm-corrected across budgets, except on the mean rows.

Dataset	Comparison	B	Δ recall	Unit CI	Units +	p
BEAM-100K and 500K	KANERVA vs hybrid	128	+0.025	[+0.010, +0.038]	34/55	0.001
BEAM-100K and 500K	KANERVA vs hybrid	1024	+0.205	[+0.184, +0.227]	55/55	<0.001
BEAM-100K and 500K	KANERVA vs hybrid	4096	+0.224	[+0.203, +0.246]	55/55	<0.001
BEAM-100K and 500K	KANERVA vs hybrid	mean	+0.127	[+0.114, +0.140]	55/55	<0.001
BEAM-100K and 500K	KANERVA vs reranker	128	+0.003	[-0.013, +0.018]	29/55	0.739
BEAM-100K and 500K	KANERVA vs reranker	1024	+0.147	[+0.121, +0.172]	51/55	<0.001
BEAM-100K and 500K	KANERVA vs reranker	4096	+0.178	[+0.156, +0.200]	55/55	<0.001
BEAM-100K and 500K	KANERVA vs reranker	mean	+0.088	[+0.072, +0.102]	52/55	<0.001
BEAM-100K and 500K	KANERVA vs first version	128	+0.012	[-0.001, +0.025]	33/55	0.128
BEAM-100K and 500K	KANERVA vs first version	1024	+0.142	[+0.120, +0.164]	52/55	<0.001
BEAM-100K and 500K	KANERVA vs first version	4096	+0.154	[+0.136, +0.174]	55/55	<0.001
BEAM-100K and 500K	KANERVA vs first version	mean	+0.086	[+0.074, +0.098]	53/55	<0.001
BEAM-100K and 500K	+ reranker vs reranker	128	+0.028	[+0.015, +0.041]	41/55	<0.001
BEAM-100K and 500K	+ reranker vs reranker	1024	+0.156	[+0.133, +0.178]	53/55	<0.001
BEAM-100K and 500K	+ reranker vs reranker	4096	+0.177	[+0.155, +0.198]	54/55	<0.001
BEAM-100K and 500K	+ reranker vs reranker	mean	+0.106	[+0.092, +0.119]	54/55	<0.001
BEAM-100K and 500K	Lexical vs BM25	128	+0.010	[-0.002, +0.023]	28/55	0.125
BEAM-100K and 500K	Lexical vs BM25	1024	+0.178	[+0.150, +0.203]	54/55	<0.001
BEAM-100K and 500K	Lexical vs BM25	4096	+0.210	[+0.186, +0.234]	55/55	<0.001
BEAM-100K and 500K	Lexical vs BM25	mean	+0.114	[+0.101, +0.128]	55/55	<0.001
RepoMem, keys	KANERVA vs hybrid	128	+0.070	[+0.027, +0.125]	5/5	0.375
RepoMem, keys	KANERVA vs hybrid	1024	-0.000	[-0.036, +0.021]	4/5	1.000
RepoMem, keys	KANERVA vs hybrid	4096	-0.025	[-0.069, +0.012]	3/5	1.000
RepoMem, keys	KANERVA vs hybrid	mean	+0.007	[-0.024, +0.028]	4/5	0.625
RepoMem, keys	KANERVA vs reranker	128	+0.055	[+0.026, +0.101]	5/5	0.375
RepoMem, keys	KANERVA vs reranker	1024	-0.003	[-0.031, +0.018]	4/5	1.000
RepoMem, keys	KANERVA vs reranker	4096	-0.025	[-0.069, +0.012]	3/5	1.000
RepoMem, keys	KANERVA vs reranker	mean	+0.004	[-0.022, +0.020]	4/5	0.875
RepoMem, keys	KANERVA vs first version	128	+0.023	[+0.007, +0.040]	4/5	0.750
RepoMem, keys	KANERVA vs first version	1024	+0.012	[+0.001, +0.025]	4/5	0.938
RepoMem, keys	KANERVA vs first version	4096	+0.011	[-0.002, +0.023]	4/5	1.000
RepoMem, keys	KANERVA vs first version	mean	+0.011	[+0.006, +0.017]	5/5	0.062
RepoMem, keys	+ reranker vs reranker	128	+0.056	[+0.030, +0.094]	5/5	0.375
RepoMem, keys	+ reranker vs reranker	1024	-0.006	[-0.033, +0.012]	3/5	1.000
RepoMem, keys	+ reranker vs reranker	4096	-0.025	[-0.069, +0.013]	3/5	1.000
RepoMem, keys	+ reranker vs reranker	mean	+0.004	[-0.023, +0.020]	4/5	0.812
RepoMem, keys	Lexical vs BM25	128	+0.071	[+0.030, +0.126]	5/5	0.375
RepoMem, keys	Lexical vs BM25	1024	-0.002	[-0.043, +0.029]	4/5	1.000
RepoMem, keys	Lexical vs BM25	4096	-0.035	[-0.072, -0.006]	0/5	0.625
RepoMem, keys	Lexical vs BM25	mean	+0.008	[-0.025, +0.035]	4/5	0.562
RepoMem, no keys	KANERVA vs hybrid	128	+0.037	[-0.003, +0.096]	4/5	1.000
RepoMem, no keys	KANERVA vs hybrid	1024	-0.014	[-0.052, +0.011]	3/5	1.000
RepoMem, no keys	KANERVA vs hybrid	4096	-0.034	[-0.082, +0.004]	2/5	1.000
RepoMem, no keys	KANERVA vs hybrid	mean	-0.009	[-0.041, +0.017]	3/5	0.812
RepoMem, no keys	KANERVA vs reranker	128	+0.024	[-0.009, +0.071]	3/5	1.000
RepoMem, no keys	KANERVA vs reranker	1024	-0.016	[-0.049, +0.011]	2/5	1.000
RepoMem, no keys	KANERVA vs reranker	4096	-0.034	[-0.082, +0.004]	2/5	1.000
RepoMem, no keys	KANERVA vs reranker	mean	-0.013	[-0.042, +0.010]	3/5	0.562
RepoMem, no keys	KANERVA vs first version	128	+0.037	[+0.012, +0.069]	4/5	0.625
RepoMem, no keys	KANERVA vs first version	1024	+0.010	[+0.003, +0.018]	5/5	0.375
RepoMem, no keys	KANERVA vs first version	4096	+0.010	[-0.003, +0.023]	4/5	0.625
RepoMem, no keys	KANERVA vs first version	mean	+0.015	[+0.010, +0.025]	5/5	0.062
RepoMem, no keys	+ reranker vs reranker	128	+0.018	[-0.008, +0.057]	4/5	1.000
RepoMem, no keys	+ reranker vs reranker	1024	-0.020	[-0.048, +0.006]	2/5	1.000
RepoMem, no keys	+ reranker vs reranker	4096	-0.035	[-0.083, +0.003]	1/5	1.000
RepoMem, no keys	+ reranker vs reranker	mean	-0.015	[-0.040, +0.007]	2/5	0.438
RepoMem, no keys	Lexical vs BM25	128	+0.047	[+0.005, +0.103]	3/5	1.000
RepoMem, no keys	Lexical vs BM25	1024	-0.017	[-0.056, +0.015]	2/5	1.000
RepoMem, no keys	Lexical vs BM25	4096	-0.044	[-0.086, -0.014]	0/5	0.375
RepoMem, no keys	Lexical vs BM25	mean	-0.009	[-0.043, +0.019]	3/5	0.750

Table 6: First held-out test, earlier version: ablations, KANERVA minus the variant with a component removed.

Dataset	Comparison	B	Δ recall	Unit CI	Units +	p
BEAM-100K and 500K	Wide pool and neighbours	1024	+0.021	[+0.008, +0.033]	36/55	0.006
BEAM-100K and 500K	Wide pool and neighbours	mean	+0.016	[+0.009, +0.022]	42/55	<0.001
BEAM-100K and 500K	Cost density	1024	+0.016	[+0.006, +0.026]	35/55	0.005
BEAM-100K and 500K	Cost density	mean	+0.016	[+0.012, +0.020]	47/55	<0.001
BEAM-100K and 500K	Validity	1024	+0.000	[-0.007, +0.007]	12/55	0.947
BEAM-100K and 500K	Validity	mean	-0.003	[-0.006, -0.000]	16/55	0.050
BEAM-100K and 500K	Near-twin rules	1024	+0.000	[-0.006, +0.007]	12/55	0.947
BEAM-100K and 500K	Near-twin rules	mean	-0.003	[-0.005, +0.000]	17/55	0.070
RepoMem, keys	Wide pool and neighbours	1024	-0.001	[-0.005, +0.002]	2/5	1.000
RepoMem, keys	Wide pool and neighbours	mean	-0.000	[-0.003, +0.003]	2/5	0.938
RepoMem, keys	Cost density	1024	-0.000	[-0.003, +0.002]	2/5	1.000
RepoMem, keys	Cost density	mean	-0.000	[-0.004, +0.003]	3/5	0.812
RepoMem, keys	Validity	1024	-0.004	[-0.038, +0.021]	3/5	1.000
RepoMem, keys	Validity	mean	+0.008	[-0.023, +0.038]	4/5	0.562
RepoMem, keys	Near-twin rules	1024	-0.009	[-0.039, +0.013]	2/5	1.000
RepoMem, keys	Near-twin rules	mean	-0.007	[-0.030, +0.008]	2/5	0.750
RepoMem, no keys	Wide pool and neighbours	1024	-0.003	[-0.008, +0.001]	1/5	1.000
RepoMem, no keys	Wide pool and neighbours	mean	-0.001	[-0.004, +0.003]	2/5	0.938
RepoMem, no keys	Cost density	1024	+0.000	[-0.005, +0.004]	3/5	1.000
RepoMem, no keys	Cost density	mean	+0.001	[-0.002, +0.004]	3/5	0.750
RepoMem, no keys	Validity	1024	-0.019	[-0.056, +0.010]	1/5	1.000
RepoMem, no keys	Validity	mean	-0.008	[-0.041, +0.023]	2/5	0.625
RepoMem, no keys	Near-twin rules	1024	-0.019	[-0.055, +0.010]	1/5	1.000
RepoMem, no keys	Near-twin rules	mean	-0.008	[-0.040, +0.022]	2/5	0.625

Table 7: First held-out test, earlier version: staleness on RepoMem queries whose fact changed, at $B = 1024$. The lexical row used vector twins (Section 6.6).

Method	Keys		No keys	
	inject	error	inject	error
KANERVA	0.000	0.000	0.194	0.000
KANERVA, lexical	0.000	0.000	0.206	0.000
First version	0.000	0.000	0.213	0.026
Hybrid, live only	0.000	0.000	0.997	0.863
Hybrid, newest of twins	0.217	0.000	0.227	0.010
Hybrid RRF	0.997	0.865	0.997	0.863
Hybrid + reranker	0.999	0.702	0.999	0.702
BM25	0.941	0.853	0.941	0.853

Table 8: Second held-out test, earlier version: evidence recall.

Method	128	256	512	1024	2048	4096	mean
RepoMem, keys							
KANERVA + reranker	**0.804**	**0.832**	**0.863**	**0.895**	**0.925**	**0.952**	**0.878**
KANERVA	0.799	0.830	0.861	0.894	0.923	0.950	0.876
KANERVA, earlier rule	0.797	0.827	0.854	0.881	0.905	0.928	0.865
KANERVA, lexical	0.758	0.812	0.846	0.879	0.907	0.924	0.854
First version	0.775	0.827	0.854	0.883	0.907	0.935	0.863
Hybrid + reranker	0.758	0.813	0.848	0.883	0.913	0.935	0.859
Hybrid RRF	0.728	0.814	0.851	0.887	0.915	0.935	0.855
Hybrid, live only	0.775	0.824	0.855	0.888	0.916	0.938	0.866
BM25	0.688	0.783	0.834	0.871	0.898	0.917	0.832
RepoMem, no keys							
KANERVA + reranker	0.757	0.801	0.837	0.873	0.905	0.934	0.851
KANERVA	0.761	0.800	0.837	0.872	0.905	**0.935**	0.852
KANERVA, earlier rule	0.760	0.798	0.830	0.860	0.887	0.913	0.841
KANERVA, lexical	0.720	0.778	0.823	0.858	0.889	0.908	0.829
First version	0.718	0.786	0.826	0.860	0.889	0.921	0.833
Hybrid + reranker	**0.763**	0.813	0.848	0.883	0.914	**0.935**	**0.859**
Hybrid RRF	0.731	**0.815**	**0.851**	**0.887**	**0.915**	**0.935**	0.856
Hybrid, live only	0.731	**0.815**	**0.851**	**0.887**	**0.915**	**0.935**	0.856
BM25	0.692	0.783	0.834	0.871	0.898	0.917	0.833

Table 9: Second held-out test, earlier version: paired differences. Conventions as in Table 5.

Dataset	Comparison	B	Δ recall	Unit CI	Units +	p
RepoMem, keys	KANERVA vs earlier rule	128	+0.002	[+0.001, +0.004]	8/8	0.047
RepoMem, keys	KANERVA vs earlier rule	1024	+0.012	[+0.004, +0.020]	6/8	0.125
RepoMem, keys	KANERVA vs earlier rule	4096	+0.021	[+0.009, +0.037]	6/8	0.125
RepoMem, keys	KANERVA vs earlier rule	mean	+0.011	[+0.005, +0.017]	7/8	0.023
RepoMem, keys	KANERVA vs hybrid	128	+0.071	[+0.054, +0.091]	8/8	0.047
RepoMem, keys	KANERVA vs hybrid	1024	+0.007	[+0.002, +0.012]	6/8	0.078
RepoMem, keys	KANERVA vs hybrid	4096	+0.015	[+0.010, +0.020]	8/8	0.047
RepoMem, keys	KANERVA vs hybrid	mean	+0.021	[+0.015, +0.028]	8/8	0.008
RepoMem, keys	KANERVA vs reranker	128	+0.041	[+0.020, +0.061]	7/8	0.078
RepoMem, keys	KANERVA vs reranker	1024	+0.010	[+0.001, +0.020]	6/8	0.078
RepoMem, keys	KANERVA vs reranker	4096	+0.015	[+0.010, +0.020]	8/8	0.047
RepoMem, keys	KANERVA vs reranker	mean	+0.018	[+0.012, +0.024]	8/8	0.008
RepoMem, keys	KANERVA vs hybrid, live	128	+0.025	[+0.014, +0.038]	8/8	0.047
RepoMem, keys	KANERVA vs hybrid, live	1024	+0.005	[-0.001, +0.011]	6/8	0.203
RepoMem, keys	KANERVA vs hybrid, live	4096	+0.012	[+0.005, +0.018]	7/8	0.094
RepoMem, keys	KANERVA vs hybrid, live	mean	+0.010	[+0.006, +0.015]	8/8	0.008
RepoMem, keys	Lexical vs BM25	128	+0.070	[+0.059, +0.084]	8/8	0.047
RepoMem, keys	Lexical vs BM25	1024	+0.008	[+0.002, +0.015]	6/8	0.094
RepoMem, keys	Lexical vs BM25	4096	+0.007	[+0.002, +0.013]	7/8	0.062
RepoMem, keys	Lexical vs BM25	mean	+0.022	[+0.016, +0.029]	8/8	0.008
RepoMem, no keys	KANERVA vs earlier rule	128	+0.001	[-0.001, +0.003]	4/8	0.484
RepoMem, no keys	KANERVA vs earlier rule	1024	+0.011	[+0.004, +0.018]	6/8	0.195
RepoMem, no keys	KANERVA vs earlier rule	4096	+0.022	[+0.008, +0.036]	6/8	0.188
RepoMem, no keys	KANERVA vs earlier rule	mean	+0.010	[+0.004, +0.016]	6/8	0.031
RepoMem, no keys	KANERVA vs hybrid	128	+0.030	[-0.002, +0.058]	6/8	0.469
RepoMem, no keys	KANERVA vs hybrid	1024	-0.015	[-0.033, -0.000]	3/8	0.586
RepoMem, no keys	KANERVA vs hybrid	4096	+0.000	[-0.012, +0.008]	6/8	0.984
RepoMem, no keys	KANERVA vs hybrid	mean	-0.004	[-0.023, +0.010]	5/8	0.867
RepoMem, no keys	KANERVA vs reranker	128	-0.002	[-0.026, +0.021]	4/8	1.000
RepoMem, no keys	KANERVA vs reranker	1024	-0.012	[-0.035, +0.006]	5/8	1.000
RepoMem, no keys	KANERVA vs reranker	4096	-0.000	[-0.012, +0.008]	6/8	1.000
RepoMem, no keys	KANERVA vs reranker	mean	-0.008	[-0.028, +0.005]	5/8	0.570
RepoMem, no keys	KANERVA vs hybrid, live	128	+0.030	[-0.003, +0.058]	6/8	0.469
RepoMem, no keys	KANERVA vs hybrid, live	1024	-0.015	[-0.034, -0.000]	3/8	0.586
RepoMem, no keys	KANERVA vs hybrid, live	4096	+0.000	[-0.012, +0.008]	6/8	0.984
RepoMem, no keys	KANERVA vs hybrid, live	mean	-0.004	[-0.021, +0.009]	5/8	0.867
RepoMem, no keys	Lexical vs BM25	128	+0.028	[+0.004, +0.050]	6/8	0.312
RepoMem, no keys	Lexical vs BM25	1024	-0.013	[-0.020, -0.005]	1/8	0.094
RepoMem, no keys	Lexical vs BM25	4096	-0.009	[-0.020, -0.001]	2/8	0.312
RepoMem, no keys	Lexical vs BM25	mean	-0.003	[-0.015, +0.006]	5/8	0.656

Table 10: Third held-out test, version 0.5.0: evidence recall. Conventions as in Table 4.

Method	128	256	512	1024	2048	4096	mean
RepoMem, keys							
KANERVA + reranker	**0.637**	**0.690**	**0.749**	**0.807**	**0.866**	**0.926**	**0.779**
KANERVA	0.630	0.689	0.744	**0.807**	0.866	**0.926**	0.777
KANERVA, lexical	0.597	0.663	0.724	0.780	0.831	0.867	0.744
KANERVA 0.4.1	0.621	0.676	0.732	0.794	0.859	0.919	0.767
KANERVA 0.4.1, lexical	0.589	0.658	0.721	0.782	0.832	0.865	0.741
Hybrid + reranker	0.591	0.658	0.722	0.781	0.843	0.899	0.749
Hybrid RRF	0.578	0.667	0.731	0.789	0.845	0.899	0.752
Hybrid, live only	0.615	0.677	0.732	0.790	0.846	0.900	0.760
BM25	0.545	0.640	0.716	0.774	0.824	0.860	0.727
RepoMem, no keys							
KANERVA + reranker	**0.630**	**0.688**	**0.746**	**0.804**	**0.865**	0.925	**0.776**
KANERVA	0.630	0.686	0.741	**0.804**	0.865	**0.925**	0.775
KANERVA, lexical	0.590	0.659	0.719	0.777	0.827	0.864	0.739
KANERVA 0.4.1	0.600	0.661	0.718	0.782	0.849	0.909	0.753
KANERVA 0.4.1, lexical	0.570	0.652	0.720	0.782	0.832	0.865	0.737
Hybrid + reranker	0.591	0.658	0.722	0.781	0.843	0.899	0.749
Hybrid RRF	0.579	0.667	0.731	0.789	0.845	0.899	0.752
Hybrid, live only	0.579	0.667	0.731	0.789	0.845	0.899	0.752
BM25	0.545	0.640	0.716	0.774	0.824	0.860	0.727

Table 11: Third held-out test, version 0.5.0: paired differences. "0.4.1" is the released method, and "any twin" drops the query condition on near-twins. Conventions as in Table 5.

Dataset	Comparison	B	Δ recall	Unit CI	Units +	p
RepoMem, keys	KANERVA vs hybrid	128	+0.051	[+0.023, +0.082]	12/15	0.008
RepoMem, keys	KANERVA vs hybrid	512	+0.013	[+0.006, +0.020]	14/15	0.002
RepoMem, keys	KANERVA vs hybrid	1024	+0.018	[+0.011, +0.025]	12/15	0.008
RepoMem, keys	KANERVA vs hybrid	4096	+0.027	[+0.019, +0.038]	14/15	0.001
RepoMem, keys	KANERVA vs hybrid	mean	+0.025	[+0.016, +0.034]	15/15	<0.001
RepoMem, keys	KANERVA vs reranker	128	+0.039	[+0.024, +0.059]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	512	+0.022	[+0.014, +0.035]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	1024	+0.026	[+0.019, +0.037]	15/15	<0.001
RepoMem, keys	KANERVA vs reranker	4096	+0.027	[+0.019, +0.037]	14/15	<0.001
RepoMem, keys	KANERVA vs reranker	mean	+0.028	[+0.021, +0.038]	15/15	<0.001
RepoMem, keys	KANERVA vs hybrid, live	128	+0.014	[+0.005, +0.024]	11/15	0.025
RepoMem, keys	KANERVA vs hybrid, live	512	+0.011	[+0.006, +0.018]	13/15	0.002
RepoMem, keys	KANERVA vs hybrid, live	1024	+0.017	[+0.010, +0.024]	12/15	0.012
RepoMem, keys	KANERVA vs hybrid, live	4096	+0.025	[+0.018, +0.035]	14/15	0.001
RepoMem, keys	KANERVA vs hybrid, live	mean	+0.017	[+0.010, +0.023]	15/15	<0.001
RepoMem, keys	KANERVA vs 0.4.1	128	+0.009	[+0.004, +0.017]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	512	+0.011	[+0.006, +0.020]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	1024	+0.012	[+0.007, +0.021]	14/15	0.001
RepoMem, keys	KANERVA vs 0.4.1	4096	+0.007	[+0.003, +0.014]	12/15	0.006
RepoMem, keys	KANERVA vs 0.4.1	mean	+0.010	[+0.006, +0.015]	14/15	<0.001
RepoMem, keys	KANERVA vs any twin	128	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	512	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	1024	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	4096	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	KANERVA vs any twin	mean	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs BM25	128	+0.052	[+0.019, +0.089]	11/15	0.063
RepoMem, keys	Lexical vs BM25	512	+0.008	[-0.003, +0.022]	10/15	0.606
RepoMem, keys	Lexical vs BM25	1024	+0.006	[+0.000, +0.011]	11/15	0.453
RepoMem, keys	Lexical vs BM25	4096	+0.007	[+0.002, +0.012]	11/15	0.606
RepoMem, keys	Lexical vs BM25	mean	+0.017	[+0.006, +0.027]	9/15	0.013
RepoMem, keys	Lexical vs 0.4.1	128	+0.009	[+0.002, +0.017]	12/15	0.036
RepoMem, keys	Lexical vs 0.4.1	512	+0.003	[-0.003, +0.007]	10/15	1.000
RepoMem, keys	Lexical vs 0.4.1	1024	-0.002	[-0.005, +0.000]	5/15	0.752
RepoMem, keys	Lexical vs 0.4.1	4096	+0.002	[+0.000, +0.005]	11/15	0.052
RepoMem, keys	Lexical vs 0.4.1	mean	+0.002	[-0.000, +0.005]	11/15	0.492
RepoMem, keys	Lexical vs any twin	128	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	512	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	1024	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	4096	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, keys	Lexical vs any twin	mean	+0.000	[+0.000, +0.000]	0/15	1.000
RepoMem, no keys	KANERVA vs hybrid	128	+0.051	[+0.022, +0.082]	12/15	0.009
RepoMem, no keys	KANERVA vs hybrid	512	+0.010	[+0.003, +0.018]	11/15	0.015
RepoMem, no keys	KANERVA vs hybrid	1024	+0.016	[+0.009, +0.023]	12/15	0.015
RepoMem, no keys	KANERVA vs hybrid	4096	+0.027	[+0.018, +0.038]	14/15	0.001
RepoMem, no keys	KANERVA vs hybrid	mean	+0.024	[+0.014, +0.033]	15/15	<0.001
RepoMem, no keys	KANERVA vs reranker	128	+0.039	[+0.023, +0.060]	14/15	<0.001
RepoMem, no keys	KANERVA vs reranker	512	+0.020	[+0.011, +0.035]	13/15	0.001
RepoMem, no keys	KANERVA vs reranker	1024	+0.024	[+0.016, +0.035]	15/15	<0.001
RepoMem, no keys	KANERVA vs reranker	4096	+0.027	[+0.018, +0.038]	14/15	<0.001
RepoMem, no keys	KANERVA vs reranker	mean	+0.026	[+0.018, +0.037]	15/15	<0.001
RepoMem, no keys	KANERVA vs hybrid, live	128	+0.051	[+0.022, +0.081]	12/15	0.009
RepoMem, no keys	KANERVA vs hybrid, live	512	+0.010	[+0.003, +0.019]	11/15	0.015
RepoMem, no keys	KANERVA vs hybrid, live	1024	+0.016	[+0.009, +0.024]	12/15	0.015
RepoMem, no keys	KANERVA vs hybrid, live	4096	+0.027	[+0.018, +0.038]	14/15	0.001
RepoMem, no keys	KANERVA vs hybrid, live	mean	+0.024	[+0.015, +0.033]	15/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	128	+0.029	[+0.019, +0.038]	14/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	512	+0.023	[+0.016, +0.030]	13/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	1024	+0.023	[+0.017, +0.030]	15/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	4096	+0.016	[+0.012, +0.021]	14/15	<0.001
RepoMem, no keys	KANERVA vs 0.4.1	mean	+0.022	[+0.017, +0.026]	15/15	<0.001
RepoMem, no keys	KANERVA vs any twin	128	+0.022	[+0.011, +0.030]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	512	+0.012	[+0.005, +0.018]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	1024	+0.011	[+0.004, +0.015]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	4096	+0.008	[+0.003, +0.012]	6/15	0.188
RepoMem, no keys	KANERVA vs any twin	mean	+0.013	[+0.006, +0.018]	6/15	0.031
RepoMem, no keys	Lexical vs BM25	128	+0.045	[+0.013, +0.080]	10/15	0.100
RepoMem, no keys	Lexical vs BM25	512	+0.003	[-0.009, +0.019]	9/15	1.000
RepoMem, no keys	Lexical vs BM25	1024	+0.002	[-0.005, +0.009]	10/15	1.000
RepoMem, no keys	Lexical vs BM25	4096	+0.003	[-0.002, +0.009]	9/15	1.000
RepoMem, no keys	Lexical vs BM25	mean	+0.013	[+0.002, +0.022]	9/15	0.032
RepoMem, no keys	Lexical vs 0.4.1	128	+0.020	[+0.007, +0.033]	13/15	0.008
RepoMem, no keys	Lexical vs 0.4.1	512	-0.000	[-0.005, +0.005]	9/15	1.000
RepoMem, no keys	Lexical vs 0.4.1	1024	-0.005	[-0.007, -0.003]	3/15	0.125
RepoMem, no keys	Lexical vs 0.4.1	4096	-0.002	[-0.004, +0.002]	9/15	1.000
RepoMem, no keys	Lexical vs 0.4.1	mean	+0.003	[-0.000, +0.005]	10/15	0.444
RepoMem, no keys	Lexical vs any twin	128	+0.031	[+0.014, +0.044]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	512	+0.033	[+0.014, +0.047]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	1024	+0.033	[+0.015, +0.047]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	4096	+0.033	[+0.014, +0.048]	6/15	0.188
RepoMem, no keys	Lexical vs any twin	mean	+0.032	[+0.015, +0.047]	6/15	0.031

Table 12: Third held-out test without keys: mean recall over budgets, by language and track. Facts exist only on Python.

Language	Track	KANERVA	Hybrid	Reranker	Lexical	BM25	0.4.1
Python	fact	**0.989**	0.977	0.985	0.969	0.955	0.968
	task	**0.514**	0.444	0.440	0.435	0.413	0.488
TypeScript	task	**0.545**	0.529	0.481	0.521	0.520	0.518
Go	task	**0.523**	0.509	0.486	0.470	0.467	0.498
Rust	task	**0.441**	0.425	0.416	0.396	0.393	0.425

Table 13: Third held-out test: staleness on queries whose fact changed, at $B = 1024$.

Method	Keys		No keys	
	inject	error	inject	error
KANERVA	0.000	0.000	0.349	0.003
KANERVA, lexical	0.000	0.000	0.266	0.005
KANERVA 0.4.1	0.000	0.000	0.352	0.003
KANERVA 0.4.1, lexical	0.000	0.000	0.955	0.007
Hybrid, newest of twins	0.365	0.000	0.365	0.003
Hybrid, live only	0.000	0.000	0.999	0.722
Hybrid RRF	0.999	0.723	0.999	0.722
Hybrid + reranker	0.999	0.596	0.999	0.596
BM25	0.990	0.834	0.990	0.834

Table 14: C and C++ test without keys: evidence recall on six repositories. Conventions as in Table 4.

Method	128	256	512	1024	2048	4096	mean
KANERVA	0.795	0.832	0.863	**0.891**	0.916	**0.940**	0.873
KANERVA + reranker	0.794	**0.836**	**0.865**	**0.891**	**0.917**	**0.940**	**0.874**
KANERVA, lexical	0.768	0.803	0.834	0.863	0.890	0.904	0.844
KANERVA 0.4.1	0.782	0.820	0.848	0.877	0.905	0.930	0.860
Hybrid + reranker	**0.798**	0.831	0.858	0.883	0.903	0.926	0.866
Hybrid RRF	0.770	0.820	0.853	0.882	0.903	0.926	0.859
Hybrid, newest of twins	0.777	0.810	0.840	0.868	0.889	0.911	0.849
BM25	0.741	0.804	0.840	0.870	0.893	0.908	0.843

Table 15: Second C and C++ test without keys, version 0.5.1: evidence recall on six repositories. Conventions as in Table 4.

Method	128	256	512	1024	2048	4096	mean
KANERVA	**0.801**	0.835	0.867	0.892	0.919	0.944	0.876
KANERVA + reranker	0.798	0.832	0.865	0.893	0.919	0.944	0.875
KANERVA, lexical	0.794	0.825	0.854	0.880	0.904	0.922	0.863
KANERVA 0.5.0	**0.801**	**0.838**	**0.870**	**0.896**	**0.923**	**0.947**	**0.879**
KANERVA 0.5.0, lexical	0.789	0.821	0.850	0.876	0.900	0.918	0.859
KANERVA 0.4.1	0.789	0.822	0.852	0.879	0.907	0.936	0.864
Hybrid + reranker	0.792	0.825	0.852	0.880	0.906	0.930	0.864
Hybrid RRF	0.791	0.832	0.857	0.880	0.905	0.930	0.866
Hybrid, newest of twins	0.795	0.827	0.850	0.873	0.897	0.922	0.860
BM25	0.770	0.823	0.851	0.873	0.895	0.916	0.855

Table 16: Third C and C++ test without keys, version 0.5.2: evidence recall on six repositories. Conventions as in Table 4.

Method	128	256	512	1024	2048	4096	mean
KANERVA	0.806	0.838	0.861	0.885	0.911	0.934	0.873
KANERVA + reranker	0.806	0.838	0.863	0.886	0.912	0.935	0.873
KANERVA, lexical	0.784	0.820	0.848	0.871	0.893	0.906	0.854
KANERVA 0.5.0	**0.815**	**0.848**	**0.870**	**0.895**	**0.922**	**0.945**	**0.882**
KANERVA 0.5.0, lexical	0.779	0.814	0.841	0.864	0.886	0.899	0.847
KANERVA 0.4.1	0.778	0.817	0.844	0.870	0.898	0.926	0.855
Hybrid + reranker	0.801	0.844	0.863	0.886	0.910	0.932	0.873
Hybrid RRF	0.757	0.846	0.868	0.889	0.910	0.932	0.867
Hybrid, newest of twins	0.783	0.835	0.855	0.875	0.893	0.911	0.859
BM25	0.721	0.829	0.862	0.884	0.903	0.916	0.852

Table 17: Query gate at 600 tokens. "Spoke" is the share of questions that got any notes. Own questions: their count, recall without and with the gate, and the share still answered with it. Foreign questions: the share that got notes without and with the gate. The third test uses twelve sampled streams and the shipped gates; other rows are out of fold.

		Own questions			Same kind, foreign		Other kind, foreign	
Set	n	recall	with gate	spoke	without	with gate	without	with gate
With vectors								
Third test	360	0.657	0.620	0.897	1.000	0.203	1.000	0.000
LongMemEval-S	12	0.979	0.896	0.917	1.000	0.440	0.957	0.137
LoCoMo	300	0.712	0.702	0.977	0.948	0.120	0.816	0.000
BEAM-1M	213	0.457	0.457	0.995	0.997	0.340	0.993	0.057
RepoMem dev	360	0.892	0.845	0.914	0.967	0.307	1.000	0.000
RepoMem holdout	360	0.853	0.846	0.964	0.987	0.423	1.000	0.000
RepoMem first test	360	0.844	0.827	0.964	0.987	0.317	0.997	0.000
RepoMem second test	360	0.852	0.833	0.950	0.983	0.357	1.000	0.000
RepoMem beyond Python	360	0.531	0.446	0.817	1.000	0.330	1.000	0.000
Lexical								
Third test	360	0.591	0.573	0.942	0.963	0.413	0.617	0.017
LongMemEval-S	12	0.833	0.833	1.000	0.987	0.900	0.987	0.783
LoCoMo	300	0.602	0.602	0.990	0.940	0.392	0.796	0.032
BEAM-1M	213	0.357	0.357	0.977	0.940	0.887	0.850	0.540
RepoMem dev	360	0.839	0.809	0.928	0.993	0.467	0.567	0.013
RepoMem holdout	360	0.816	0.806	0.961	1.000	0.540	0.637	0.043
RepoMem first test	360	0.829	0.787	0.925	0.960	0.367	0.510	0.003
RepoMem second test	360	0.816	0.788	0.950	0.987	0.517	0.693	0.040
RepoMem beyond Python	360	0.457	0.411	0.883	0.973	0.477	0.587	0.013

Table 18: File notes with vectors. Recall on the task track from the prompt alone, from the prompt with the same tokens added, and from the prompt plus file notes. Precision is that of the notes, and the last columns count repositories where notes beat equal tokens or lose to them.

Set	Session	Prompt	Same tokens	With notes	Precision	Better	Worse
Third test	this task	0.430	0.463	0.560	0.289	15	0
Third test	last task	0.430	0.478	0.445	0.024	0	15
Beyond Python	this task	0.457	0.480	0.555	0.272	9	0
Beyond Python	last task	0.453	0.494	0.466	0.021	0	9
Second test	this task	0.432	0.466	0.520	0.242	7	1
Second test	last task	0.427	0.482	0.440	0.020	0	8
First test	this task	0.366	0.389	0.423	0.261	4	1
First test	last task	0.365	0.403	0.379	0.055	1	4
Holdout	this task	0.451	0.477	0.531	0.250	4	0
Holdout	last task	0.452	0.500	0.463	0.028	0	4
RepoMem dev	this task	0.467	0.497	0.603	0.282	8	0
RepoMem dev	last task	0.453	0.499	0.481	0.037	2	6

Table 19: The write path without keys. Changed notes, how often the older version is first or anywhere on the list of up to five, how often a list appears on notes that replace nothing, stale injection at $B = 1024$ now and with a simulated oracle agent, and the tokens of a list.

Set	Changed	First	Listed	List on others	Stale	Stale, oracle	Tokens
With vectors							
RepoMem, third test	1,904	0.931	0.994	1.000	0.210	0.003	154
RepoMem, second test	5,279	0.939	0.998	1.000	0.183	0.003	152
RepoMem, first test	841	0.910	0.993	0.999	0.181	0.007	137
LongMemEval updates	47	0.511	0.809	0.926	1.000	0.319	200
Lexical							
RepoMem, third test	1,904	0.898	0.952	0.530	0.290	0.035	83
RepoMem, second test	5,279	0.930	0.984	0.612	0.264	0.031	92
RepoMem, first test	841	0.895	0.975	0.482	0.268	0.028	97
LongMemEval updates	47	0.511	0.723	0.373	0.979	0.340	102

Table 20: Prequential recall at $B = 512$, mean ± s.d. over five seeds, on simulated feedback. "Noisy" flips 20% of rewards. The bandit is the analysed rule, lift the plugins' opt-in rule.

Dataset	Static	Bandit	Bandit, noisy	Lift	Lift, noisy
LongMemEval-S	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000	0.906 ± 0.000
LoCoMo	0.711 ± 0.001	0.668 ± 0.011	0.683 ± 0.010	0.717 ± 0.003	0.710 ± 0.002
RepoMem	0.902 ± 0.002	0.885 ± 0.006	0.895 ± 0.004	0.902 ± 0.002	0.901 ± 0.002

Table 21: Share of answers that contain the reference, from the notes a method selected, at 256, 512 and 1024 tokens. "Labelled evidence" hands the reader the gold notes alone. With no notes the score is zero.

Method	LoCoMo			LongMemEval-S			BEAM-1M			RepoMem, third test		
	256	512	1024	256	512	1024	256	512	1024	256	512	1024
Kanerva	0.267	0.333	0.350	0.467	0.483	0.483	0.275	0.300	0.375	0.911	0.956	0.900
Kanerva, lexical	0.250	0.283	0.350	0.433	0.500	0.467	0.275	0.350	0.400	0.833	0.889	0.867
Kanerva 0.4.1	0.300	0.317	0.333	0.500	0.550	0.433	0.250	0.400	0.450	0.856	0.878	0.900
Kanerva 0.4.1, lexical	0.233	0.333	0.350	0.467	0.500	0.483	0.300	0.325	0.325	0.833	0.856	0.844
Hybrid RRF	0.233	0.233	0.283	0.367	0.417	0.450	0.350	0.400	0.325	0.867	0.856	0.856
BM25	0.283	0.350	0.350	0.517	0.483	0.450	0.375	0.350	0.350	0.856	0.844	0.811
Labelled evidence		0.467			0.683			0.375			0.922	

Table 22: Answers: paired differences of the mean over 256, 512 and 1024 tokens. The unit is a conversation or a repository, and a question on LongMemEval-S, so its positive-unit count is omitted. p is raw.

Dataset	Comparison	Δ	Interval	Units +	p
LoCoMo	Kanerva vs hybrid	+0.067	[-0.017, +0.144]	7/10	0.199
LoCoMo	Kanerva vs BM25	-0.011	[-0.078, +0.044]	5/10	0.875
LoCoMo	Lexical vs BM25	-0.033	[-0.089, +0.011]	3/10	0.375
LoCoMo	Kanerva vs 0.4.1	+0.000	[-0.028, +0.028]	3/10	1.000
LoCoMo	Lexical vs 0.4.1	-0.011	[-0.044, +0.017]	3/10	0.750
LongMemEval-S	Kanerva vs hybrid	+0.067	[+0.000, +0.139]	—	0.103
LongMemEval-S	Kanerva vs BM25	-0.006	[-0.078, +0.067]	—	1.000
LongMemEval-S	Lexical vs BM25	-0.017	[-0.072, +0.039]	—	0.716
LongMemEval-S	Kanerva vs 0.4.1	-0.017	[-0.061, +0.022]	—	0.612
LongMemEval-S	Lexical vs 0.4.1	-0.017	[-0.044, +0.011]	—	0.453
BEAM-1M	Kanerva vs hybrid	-0.042	[-0.128, +0.037]	4/35	0.427
BEAM-1M	Kanerva vs BM25	-0.042	[-0.111, +0.035]	4/35	0.400
BEAM-1M	Lexical vs BM25	-0.017	[-0.075, +0.044]	3/35	1.000
BEAM-1M	Kanerva vs 0.4.1	-0.050	[-0.117, +0.000]	1/35	0.186
BEAM-1M	Lexical vs 0.4.1	+0.025	[-0.017, +0.067]	5/35	0.380
RepoMem, third test	Kanerva vs hybrid	+0.063	[+0.007, +0.154]	4/6	0.125
RepoMem, third test	Kanerva vs BM25	+0.085	[+0.045, +0.144]	6/6	0.031
RepoMem, third test	Lexical vs BM25	+0.026	[-0.051, +0.100]	4/6	0.375
RepoMem, third test	Kanerva vs 0.4.1	+0.044	[-0.008, +0.095]	5/6	0.156
RepoMem, third test	Lexical vs 0.4.1	+0.019	[-0.051, +0.086]	3/6	0.781

Table 23: Answers on facts that changed, third test: percent of answers that give the old value, and of queries with a stale note in the context.

Method	Old value answered			Stale note injected		
	256	512	1024	256	512	1024
Kanerva	0.0	0.0	5.7	37.1	40.0	40.0
Kanerva, lexical	0.0	2.9	5.7	45.7	45.7	45.7
Kanerva 0.4.1	2.9	5.7	2.9	40.0	42.9	42.9
Kanerva 0.4.1, lexical	25.7	17.1	17.1	94.3	94.3	97.1
Hybrid RRF	8.6	11.4	14.3	97.1	100.0	100.0
BM25	14.3	14.3	20.0	97.1	97.1	100.0

Table 24: Reversible consolidation (LongMemEval-S: 60 random questions). τ is the similarity threshold, R the extractive rate.

Dataset, τ, rate	Clusters	Sources	Tokens saved	Recall@1024
LoCoMo, baseline	0	0	0.0%	0.824
LoCoMo, $\tau = 0.7$, $R = 0.5$	577	1972	23.0%	0.669
LoCoMo, $\tau = 0.8$, $R = 0.5$	195	535	5.6%	0.791
LoCoMo, $\tau = 0.9$, $R = 0.5$	29	71	0.5%	0.823
LongMemEval-S, baseline	0	0	0.0%	0.969
LongMemEval-S, $\tau = 0.7$, $R = 0.5$	7985	20806	39.4%	0.689
LongMemEval-S, $\tau = 0.8$, $R = 0.5$	4625	10327	19.9%	0.789
LongMemEval-S, $\tau = 0.9$, $R = 0.5$	458	1015	2.1%	0.969

Table 25: Forgetting on RepoMem streams: final active store and recall.

Policy	Active items	Active tokens	Recall@1024
keep-all	965	21492	0.923
idle-730d	474	11661	0.857
idle-365d	243	6176	0.713
idle-180d	130	3227	0.551
cap-8k	426	7986	0.750
cap-4k	239	3989	0.540

7 Limitations

Answers rest on one small reader. Most metrics are retrieval-level. The only answer-level evidence is Section 6.17: one small local model, short extractive questions and small samples. It found no reliable difference between methods, and its intervals are wide, so it does not show that methods are equal. A stronger model may use the notes differently, and a coding agent does more than answer short questions. Evidence recall treats a turn as all-or-nothing, and a selected turn may still be hard for a model to use. How much recall per token turns into finished tasks is unknown, and that is the next measurement.

Baselines that need an LLM were not run. Mem0 with inference, Graphiti, A-MEM, HippoRAG, LangMem and Cognee all call an LLM on write, so the comparison does not cover them. Mem0 ran only in its verbatim mode (`infer=False`), on a subset. Our comparison says nothing about what LLM extraction would add to those systems, and published evidence suggests a small effect on the write side [70].

Generalisation. The relevance model is fitted on four benchmark families, with BEAM-1M as the only source of long conversations, so the BEAM results are about new conversations from a family it has seen. BEAM-100K and 500K have been seen by earlier tests as well, and no untouched conversation benchmark is left. Every set of repositories is spent too once it has been read, and the third test's fifteen now are. On RepoMem the gain is a few points, against far more on BEAM. The fact track reads Python, and C and C++ by regular expressions, of which only the last test reads templates, overloads and class methods, so nothing here says how exact code facts or their staleness behave in TypeScript, Go or Rust. Each of those languages rests on three repositories, and C and C++ on nine each. A deployment on notes unlike any of these may behave differently, and the features assume turn structure that real stores may lack. LongMemEval-S and LoCoMo informed the design of the method and are not clean. Fitting chats and code at equal weight cost LoCoMo a little, out of fold.

Validity evidence is narrow. The near-twin rule works on RepoMem because versions differ only in a value, and it does not fire on natural-language updates. Without keys it still lets an outdated version through 34.9% of the time with MiniLM and 26.6% lexically on the third test, against none with keys. It costs some recall on exact facts without keys, more for lexical stores, and it relies on the `kind` a note was written with. The hazard model has no ablation that isolates it, and detection of historical intent was not evaluated on any benchmark.

The gate. It costs recall on a store's own questions, 5.6% with vectors and 3.0% lexically on the third test and up to 16.0% out of fold, and those queries get nothing at all when it closes. Its false-alarm numbers come from foreign questions, which are easier to reject than vague prompts about the same project, and on twelve sampled streams. On one developer's real prompts it separates far less (Section 6.11), and the file-overlap label there is weak. How often it is wrongly silent on real prompts is still not known exactly. Session start is not gated.

File notes. Task labels come from file overlap, so the gain when the file belongs to the task is close to true by construction, and the loss when it belongs to the last task is a worst case. Real sessions were not measured. Delivery was checked live on Claude Code with a real model, and on Codex and Copilot CLI with a stub model.

The write path. The upper bound is an oracle that stands in for the host agent, so it is simulated. A real agent was measured on one model and a few dozen notes, with the hint, the server's instructions and the list set after earlier rounds. It fixed most changed facts, and it sometimes retired more than it should. Another agent or model may do otherwise, and a store without vectors lists far fewer paraphrases. The list also appears on many notes that replace nothing, and each appearance costs the agent tokens.

Feedback was simulated. The online experiments derive rewards from evidence labels. Real implicit signals, such as an item's identifiers reappearing in tool calls, are noisier and biased in ways that label flipping does not capture. Replayed on real sessions the plugin's signal fires on many notes the agent never saw (Section 6.11), so the small gain of item lift on LoCoMo says nothing about real use.

RepoMem is a proxy. Queries come from templates over extracted facts, so they contain identifiers and favour lexical search; real prompts are vaguer. The agent's notes are simulated from code and commit history. Every changed fact is assumed to be observed, so a store never misses a change. The task track uses file overlap as relevance, which is a weak label. Keyed mode gets perfect slot keys from the static analyzer, so it is an upper bound on what a real write path provides; plain mode is the conservative end. Some changed facts only moved to another file, and the dependency extractor misreads some console-script entries. Section 6.6 measures both on the third test, and the earlier tests have the same flaws.

Answer-term recall. It counts word stems, so it misses paraphrase, and it can credit a word found in an unrelated item. It serves here to compare excerpts with full items, not as a measure of answerability.

Theory versus code. Theorem 2 assumes a linear μ, conditionally sub-Gaussian rewards and validity revealed with feedback. In practice validity is seen only when a successor appears, so unkeyed items that are stale enter learning with a reward of zero. As stated in Section 5, the bound is about scaling, and the defaults differ from the analyzed algorithm. The theory analyses the linear bandit, while the plugins' opt-in path is the item lift of Section 4, which no result here covers. Hazards assume exponential lifetimes within a class. The certificate bounds the surrogate F, not recall, and Proposition 7 bounds expected hits under the model's probabilities, which are calibrated only on data like the training data. Theorem 1 covers selection before the plugins' threshold and gate, not after them. Excerpts have no guarantee.

Plugins. The benchmarks ran the library with exact token counts. The plugins estimate tokens with a dependency-free approximation. Their tokenizer also handles non-Latin scripts; on a sample of held-out queries this changed fewer than one row in 300, by less than a thousandth of recall on average. Until the model arrives, a new install runs the lexical model, so its first sessions get the lexical numbers. Beyond English, only Russian was measured, on XQuAD and not on coding notes.

Token counts. We count with `cl100k_base` as a proxy for Claude's tokenizer, and the 14-token item overhead is an approximation of the rendered prefix.

8 Conclusion

Treating agent memory as budgeted selection over items with explicit validity gives a read path that needs no LLM, and a theory in which expiring items have a clear price. The strongest practical lever turned out to be simpler than the theory: a calibrated estimate of what each candidate is worth. On a test of fifteen repositories in four languages, the current release beats hybrid retrieval and a cross-encoder on every repository when averaged over budgets, with or without keys, by a few points of recall. Three tests on six C and C++ repositories each show a smaller lead, which does not hold on the third once class methods count as facts, and a rule that compares near-twins both by vectors and by words cuts stale injection to about a third. On long conversations the lead is far larger, but no untouched conversation benchmark is left to confirm it. Stale injection is a large problem on code facts, and simple rules fix most of it, so the probabilistic validity model has not yet shown value beyond them. A small local model answering from the selected notes showed no reliable difference between methods, so the recall gains are not yet shown to reach answers, though stale notes rarely became stale answers. Free-text updates need a host agent: on its own the store cannot tell that one replaces another, but a real agent shown similar notes fixed most of them. Stale injection in stores without vectors had been understated until the benchmark was corrected. On one developer's real prompts the gate separates far less than on the benchmarks and the implicit use signal fires on notes nobody saw, so online learning, which adds almost nothing on top of the fitted model, stays opt-in.

The next step is an end-to-end study: whether higher recall per token, and fewer stale injections, change what a coding agent does. The retrieval results here are necessary for that outcome and do not establish it.

Reproducibility. `make data && make reproduce` rebuilds the current results in `results/*.json`, and `make check` verifies each number in this paper against those files. The first two held-out tests are the record of the earlier version: their result files name the commit that produced them, and rerunning them scores the current method, so they run only on request. The RepoMem manifest pins every repository snapshot.

References

[1] Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved Algorithms for Linear Stochastic Bandits. In *Advances in Neural Information Processing Systems 24*, 2011. URL `https://proceedings.neurips.cc/paper/2011/hash/e1d5be1c7f2f456670de3d53c7b54f4a-Abstract.html`. Accessed 2026-10-30.

[2] John R. Anderson and Lael J. Schooler. Reflections of the Environment in Memory. *Psychological Science*, 2:396–408, 1991. doi: 10.1111/j.1467-9280.1991.tb00174.x. URL `https://doi.org/10.1111/j.1467-9280.1991.tb00174.x`. Accessed 2026-10-30.

[3] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An Integrated Theory of the Mind. *Psychological Review*, 111:1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036. URL `https://doi.org/10.1037/0033-295X.111.4.1036`. Accessed 2026-10-30.

[4] Anthropic. How Claude remembers your project, 2026. URL https://code.claude.com/docs/en/memory. Official docs. Accessed 2026-10-30.

[5] Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the Cross-lingual Transferability of Monolingual Representations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4623–4637, 2020. doi: 10.18653/v1/2020.acl-main.421. URL https://doi.org/10.18653/v1/2020.acl-main.421. Introduces XQuAD; checked against arXiv, the ACL Anthology and the dataset repository. Accessed 2026-10-31.

[6] Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory Reward Inflation in Self-Improving LLM Agents, 2026. URL https://arxiv.org/abs/2608.00017. Accessed 2026-10-30.

[7] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to Memorize at Test Time, 2024. URL https://arxiv.org/abs/2501.00663. Accessed 2026-10-30.

[8] Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory Layers at Scale, 2024. URL https://arxiv.org/abs/2412.09764. Accessed 2026-10-30.

[9] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025. Accessed 2026-10-30.

[10] Wei Chen, Yajun Wang, and Yang Yuan. Combinatorial Multi-Armed Bandit: General Framework and Applications. In *International Conference on Machine Learning*, volume 28 of *PMLR*, pages 151–159, 2013. URL https://proceedings.mlr.press/v28/chen13a.html. Accessed 2026-10-30.

[11] Wei Chen, Yajun Wang, Yang Yuan, and Qinshi Wang. Combinatorial Multi-Armed Bandit and Its Extension to Probabilistically Triggered Arms, 2014. URL https://arxiv.org/abs/1407.8339. Accessed 2026-10-30.

[12] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting Language Models to Compress Contexts, 2023. URL https://arxiv.org/abs/2305.14788. Accessed 2026-10-30.

[13] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025. URL https://arxiv.org/abs/2504.19413. Accessed 2026-10-30.

[14] Ashwin Gerard Colaco and Nada Lahjouji. What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents, 2026. URL https://arxiv.org/abs/2607.08032. Accessed 2026-10-30.

[15] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114. Accessed 2026-10-30.

[16] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From Local to Global: A Graph RAG Approach to Query-Focused Summarization, 2024. URL https://arxiv.org/abs/2404.16130. Accessed 2026-10-30.

[17] Liyang Fan, Yingcheng Shi, Yongbin Li, Chenghao Sun, Xin Chen, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. VibeMemBench: Evaluating Memory Systems for Coding Agents on Real Repository Coding Tasks, 2026. URL https://arxiv.org/abs/2609.23570. Accessed 2026-10-30.

[18] Moran Feldman, Zeev Nutov, and Elad Shoham. Practical Budgeted Submodular Maximization, 2020. URL https://arxiv.org/abs/2007.04937. Accessed 2026-10-30.

[19] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context Autoencoder for Context Compression in a Large Language Model, 2023. URL https://arxiv.org/abs/2307.06945. Accessed 2026-10-30.

[20] Manu Ghulyani, Arunabh Singh, Karan Bharadwaj, Ankit Nath, and Suranjan Goswami. PACMS: Submodular Context Selection as a Pluggable Engine for LLM Agents, 2026. URL https://arxiv.org/abs/2606.20047. Accessed 2026-10-30.

[21] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and Fast Retrieval-Augmented Generation, 2024. URL https://arxiv.org/abs/2410.05779. Accessed 2026-10-30.

[22] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.14802. Accessed 2026-10-30.

[23] Yudai Honda, Keita Fujita, Keiichi Zempo, and Shogo Fukushima. Human-Like Remembering and Forgetting in LLM Agents: An ACT-R-Inspired Memory Architecture. In *International Conference on Human-Agent Interaction*, pages 229–237, 2025. doi: 10.1145/3765766.3765803. URL https://doi.org/10.1145/3765766.3765803. Accessed 2026-10-30.

[24] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2507.05257. Year refers to the ICLR 2026 publication. Accessed 2026-10-30.

[25] Mehmet Iscan. PYTHALAB-MERA: Validation-Grounded Memory, Retrieval, and Acceptance Control for Frozen-LLM Coding Agents, 2026. URL https://arxiv.org/abs/2605.08468. Accessed 2026-10-30.

[26] Mehmet Iscan. Learning When to Remember: Risk-Sensitive Contextual Bandits for Abstention-Aware Memory Retrieval in LLM-Based Coding Agents, 2026. URL https://arxiv.org/abs/2604.27283. Accessed 2026-10-30.

[27] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2310.05736. Accessed 2026-10-30.

[28] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression, 2023. URL https://arxiv.org/abs/2310.06839. Accessed 2026-10-30.

[29] Prakhar Khatri. Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories, 2026. URL https://arxiv.org/abs/2607.27250. Accessed 2026-10-30.

[30] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70: 39–45, 1999. doi: 10.1016/S0020-0190(99)00031-9. URL https://doi.org/10.1016/S0020-0190(99)00031-9. Accessed 2026-10-30.

[31] Andreas Krause and Carlos Guestrin. A Note on the Budgeted Maximization of Submodular Functions. Technical Report CMU-CALD-05-103, Carnegie Mellon University, 2005. URL http://reports-archive.adm.cs.cmu.edu/anon/cald/CMU-CALD-05-103.pdf. Accessed 2026-10-30.

[32] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning, 2012. URL https://arxiv.org/abs/1207.6083. Accessed 2026-10-30.

[33] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. *Trends in Cognitive Sciences*, 20:512–534, 2016. doi: 10.1016/j.tics.2016.05.004. URL https://doi.org/10.1016/j.tics.2016.05.004. Accessed 2026-10-30.

[34] Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvari. Tight Regret Bounds for Stochastic Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1410.0949. Accessed 2026-10-30.

[35] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 420–429, 2007. doi: 10.1145/1281192.1281239. URL https://doi.org/10.1145/1281192.1281239. Accessed 2026-10-30.

[36] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A Benchmark for Context Retrieval in Coding Agents, 2026. URL https://arxiv.org/abs/2602.05892. Accessed 2026-10-30.

[37] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM Knows What You are Looking for Before Generation, 2024. URL https://arxiv.org/abs/2404.14469. Accessed 2026-10-30.

[38] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL

https://arxiv.org/abs/2402.17753. Accessed 2026-10-30.

[39] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102:419–457, 1995. doi: 10.1037/0033-295X.102.3.419. URL https://doi.org/10.1037/0033-295X.102.3.419. Accessed 2026-10-30.

[40] Michel Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, Lecture Notes in Control and Information Sciences, pages 234–243. 1978. doi: 10.1007/BFb0006528. URL https://doi.org/10.1007/BFb0006528. Accessed 2026-10-30.

[41] Kazuki Nakayashiki. When Stale Constraints Go Unchecked: Budgeted Verification Failures in Inherited Agent Memory, 2026. URL https://arxiv.org/abs/2608.25553. Accessed 2026-10-30.

[42] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971. URL https://doi.org/10.1007/BF01588971. Accessed 2026-10-30.

[43] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, 2025. URL https://arxiv.org/abs/2509.25140. Accessed 2026-10-30.

[44] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://arxiv.org/abs/2310.08560. Accessed 2026-10-30.

[45] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression, 2024. URL https://arxiv.org/abs/2403.12968. Accessed 2026-10-30.

[46] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL https://arxiv.org/abs/2304.03442. UIST 2023; DOI 10.1145/3586183.3606763. Accessed 2026-10-30.

[47] Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. AdaGReS:Adaptive Greedy Context Selection via Redundancy-Aware Scoring for Token-Budgeted RAG, 2025. URL https://arxiv.org/abs/2512.25052. Accessed 2026-10-30.

[48] Aleksandar Petrov, Mark Sandler, Andrey Zhmoginov, Nolan Miller, and Max Vladymyrov. Long Context In-Context Compression by Getting to the Gist of Gisting, 2025. URL https://arxiv.org/abs/2504.08934. Accessed 2026-10-30.

[49] Nathanaël Carraz Rakotonirina, Mohammed Hamdy, Jon Ander Campos, Lucas Weber, Alberto Testoni, Marzieh Fadaee, Sandro Pezzelle, and Marco Del Tredici. From Tools to Teammates: Evaluating LLMs in Multi-Session Coding Interactions, 2025. URL https://arxiv.org/abs/2502.13791. Accessed 2026-10-30.

[50] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025. URL https://arxiv.org/abs/2501.13956. Accessed 2026-10-30.

[51] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Text REtrieval Conference (TREC-3)*, pages 109–126, 1994. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/okapi_trec3.pdf. Accessed 2026-10-30.

[52] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval, 2024. URL https://arxiv.org/abs/2401.18059. Accessed 2026-10-30.

[53] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL https://arxiv.org/abs/2303.11366. Accessed 2026-10-30.

[54] Baixi Sun, Le Chen, Anjir Ahmed Chowdhury, Xiaolong Ma, Chih-Hsuan Yang, Mingze Xia, Syed Zawad, Sheng Di, Rajkumar Kettimuthu, Huihuo Zheng, Rajeev Thakur, Venkatram Vishwanath, and Feng Yan. LAM: Efficient Lossy Agent

Memory Framework With A Retrieval-Score Error Bound, 2026. URL https://arxiv.org/abs/2609.32256. Accessed 2026-10-30.

[55] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts, 2019. URL https://arxiv.org/abs/1909.13231. Accessed 2026-10-30.

[56] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (Learn at Test Time): RNNs with Expressive Hidden States, 2024. URL https://arxiv.org/abs/2407.04620. Accessed 2026-10-30.

[57] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32:41–43, 2004. doi: 10.1016/S0167-6377(03)00062-2. URL https://doi.org/10.1016/S0167-6377(03)00062-2. Accessed 2026-10-30.

[58] Sho Takemori, Masahiro Sato, Takashi Sonoda, Janmajay Singh, and Tomoko Ohkuma. Submodular Bandit Problem Under Multiple Constraints, 2020. URL https://arxiv.org/abs/2006.00661. PMLR 124 (UAI 2020). Accessed 2026-10-30.

[59] Jing Tang, Xueyan Tang, Andrew Lim, Kai Han, Chongshou Li, and Junsong Yuan. Revisiting Modified Greedy Algorithm for Monotone Submodular Maximization with a Knapsack Constraint, 2020. URL https://arxiv.org/abs/2008.05391. Accessed 2026-10-30.

[60] Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts, 2026. URL https://arxiv.org/abs/2605.20926. Accessed 2026-10-30.

[61] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.27246. Accessed 2026-10-30.

[62] Sentence Transformers. sentence-transformers/all-MiniLM-L6-v2 model card, 2026. URL https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Model card. Accessed 2026-10-30.

[63] Stephan Tulkens and Thomas van Dongen. Model2Vec: Fast State-of-the-Art Static Embeddings, 2024. URL https://github.com/MinishLab/model2vec. Repository. Accessed 2026-10-30.

[64] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL https://arxiv.org/abs/2305.16291. Accessed 2026-10-30.

[65] Zheng Wen, Branislav Kveton, and Azin Ashkan. Efficient Learning in Large-Scale Combinatorial Semi-Bandits, 2014. URL https://arxiv.org/abs/1406.7443. Accessed 2026-10-30.

[66] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10813. Accessed 2026-10-30.

[67] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks, 2023. URL https://arxiv.org/abs/2309.17453. Accessed 2026-10-30.

[68] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents, 2025. URL https://arxiv.org/abs/2502.12110. Accessed 2026-10-30.

[69] Baosheng Yu, Meng Fang, and Dacheng Tao. Linear Submodular Bandits with a Knapsack Constraint. In *AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi: 10.1609/aaai.v30i1.10154. URL https://doi.org/10.1609/aaai.v30i1.10154. Accessed 2026-10-30.

[70] Boqin Yuan, Yue Su, and Kun Yao. Diagnosing Retrieval vs. Utilization Bottlenecks in LLM Agent Memory, 2026. URL https://arxiv.org/abs/2603.02473. Accessed 2026-10-30.

[71] Yisong Yue and Carlos Guestrin. Linear Submodular Bandits and their Application to Diversified Retrieval. In *Advances in Neural Information Processing Systems 24*, pages 2483–2491, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/33ebd5b07dc7e407752fe773eed20635-Abstract.html. Accessed 2026-10-30.

[72] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models, 2025. URL https://arxiv.org/abs/2510.04618. Accessed 2026-10-30.

[73] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory, 2026. URL https://arxiv.org/abs/2601.03192. Accessed 2026-10-30.

[74] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models, 2023. URL https://arxiv.org/abs/2306.14048. Accessed 2026-10-30.

[75] Ziyi Zhang, Shuang Cui, Haotian Zhang, and Xiaoyu Wang. PC-SubMax: Efficient Prompt Compression via Regularized Submodular Maximization, 2026. URL https://arxiv.org/abs/2609.32474. Accessed 2026-10-30.

[76] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A Benchmark for Context Learning in Coding, 2026. URL https://arxiv.org/abs/2602.08316. Accessed 2026-10-30.

[77] Mingxi Zou, Zhihan Guo, Langzhang Liang, Zhuo Wang, Qifan Wang, Qingsong Wen, Irwin King, Lizhen Qu, and Zenglin Xu. Remember the Decision, Not the Description: A Rate-Distortion Framework for Agent Memory, 2026. URL https://arxiv.org/abs/2605.10870. Accessed 2026-10-30.

A Proofs

Proof of Lemma 1. For a facet f, $g_f(S) = 1 - \prod_{m \in S}(1 - u_m)$ with $u_m = w(m)a_{mf} \in [0, 1]$. Adding m to S gains $u_m \prod_{n \in S}(1-u_n)$, which is non-negative and non-increasing in S. So g_f is monotone submodular. The additive term is modular with non-negative coefficients. Each term $S \mapsto \max_{m \in S} s(c, m)$ (with $\max \emptyset = 0$) is the facility-location function, which is monotone submodular. Non-negative combinations preserve both properties, and $F(\emptyset) = 0$.

Coordinate monotonicity follows because $\prod(1 - w(m)a_{mf})$ decreases in each $w(m)$ and the third term does not depend on w. For the Lipschitz bound, $|\prod_i(1 - u_i) - \prod_i(1 - v_i)| \le \sum_i |u_i - v_i|$ for $u, v \in [0, 1]$ by telescoping. So each g_f moves by at most $\sum_{m \in S} |w(m) - w'(m)|$ as $a_{mf} \le 1$, and $\sum_f \omega_f = 1$. The additive term moves by at most η times the same sum, and the factor $(1 - \gamma)$ gives L. □

Proof of Proposition 1. Take additive U. One item a has score $1 + \epsilon$ and cost B. There are $\lfloor B/c_{\min} \rfloor$ items of score 1 and cost $c_{\min}$. The rule fills with a first and then has no budget, so it earns $1 + \epsilon$ against OPT $\ge \lfloor B/c_{\min} \rfloor$. Let $\epsilon \to 0$. □

Proof of Proposition 2. Let there be k facets of equal weight. Facet 1 has k duplicates, each with modular score $1 + \epsilon$. Each other facet has one item with score 1. All costs equal 1, and $B = k$. A modular rule takes the k duplicates and covers one facet; the optimum takes one item per facet and covers all k. □

Proof of Proposition 3. Walking from newest to oldest, keep each item that fits. Since the later items cost more than B in total, one of them fails to fit, and at that moment the remaining budget is below its cost, which is at most $c(m)$. The remaining budget only shrinks afterwards, so m never fits. A relevance rule does not depend on the items' order in time. □

Proof of Proposition 4. Take two versions with equal text similarity to the query, and let the assignment of the two timestamps be uniform. An invariant score cannot depend on the assignment, so the ranking between the two is independent of which one is current. The outdated version ranks first with probability 1/2, or at least 1/2 under ties broken independently of time. □

Proof of Proposition 5. Let each round have two items with features e_1, e_2, costs 1, $B = 1$, $\rho \equiv 1$ and additive F. For a fixed θ, assume without loss of generality that $\theta_1 \ge \theta_2$; the fixed policy injects e_1 on every round. Take $\theta^\star = (0, 1)$, which is valid since $\mu \in [0, 1]$. The policy earns 0 per round against 1, so its α-regret is αT for each $\alpha > 0$. Theorem 2 with $\varepsilon \equiv 0$ gives $\tilde{O}(\sqrt{T})$ on this instance. □

Proof of Theorem 1. Let O be an optimal set and consider the density run ($\varrho = 1$) as a sequence of considered items. Let $g_1, g_2, \dots$ be the items accepted, in order, and write $G_i = \{g_1, \dots, g_i\}$. If no element of O is rejected, then every element of O is either accepted or left with zero marginal gain at termination, so $F(G) \ge F(G \cup O) \ge F(O)$ and we are done. Otherwise let $e^\star$ be the first element of O rejected, and let ℓ be the number of accepted items before it, so $c(G_\ell) + c(e^\star) > B$.

For $i \le \ell$, when g_i was chosen, every element of $O \setminus G_{i-1}$ was still available, since none had been rejected. By monotonicity and submodularity,

$$F(O) \le F(O \cup G_{i-1}) \le F(G_{i-1}) + \sum_{o \in O \setminus G_{i-1}} \Delta(o \mid G_{i-1}) \le F(G_{i-1}) + B\,\frac{\Delta(g_i \mid G_{i-1})}{c(g_i)}.$$

The same inequality holds with $e^\star$ in the role of $g_{\ell+1}$ and $G_{\ell+1} := G_\ell \cup \{e^\star\}$, $c(g_{\ell+1}) := c(e^\star)$, since $e^\star$ had maximum density when chosen. Set $\delta_i = F(O) - F(G_i)$. The inequality reads $\delta_i \le (1 - c(g_i)/B)\,\delta_{i-1}$. If some $\delta_i \le 0$ with $i \le \ell$ then $F(G) \ge F(O)$ and we are done. Otherwise, using $0 \le 1 - x \le e^{-x}$ on $x = c(g_i)/B \in (0, 1]$,

$$\delta_{\ell+1} \le F(O) \prod_{i=1}^{\ell+1} \Big(1 - \frac{c(g_i)}{B}\Big) \le F(O) \exp\Big(-\frac{\sum_{i \le \ell+1} c(g_i)}{B}\Big) \le \frac{F(O)}{e},$$

because $\sum_{i \le \ell+1} c(g_i) = c(G_\ell) + c(e^\star) > B$. So $F(G_\ell \cup \{e^\star\}) \ge (1 - 1/e)F(O)$. By subadditivity $F(G_\ell) + F(\{e^\star\}) \ge (1 - 1/e)F(O)$, so one of the two terms is at least $\frac{1}{2}(1 - 1/e)F(O)$. Now $F(G) \ge F(G_\ell)$ by monotonicity, and $F(\{e^\star\})$ is at most the value of the best singleton, which fits since $c(e^\star) \le B$. The algorithm returns the maximum over a set of runs that includes both. □

Proof of Proposition 6. For any feasible O, monotonicity and submodularity give $F(O) \le F(S) + \sum_{m \in O \setminus S} \Delta(m \mid S)$. The indicator of $O \setminus S$ is feasible for the linear program, whose optimum is reached by the fractional-knapsack rule. □

Lemma 2 (Batched elliptical potential)**.** *Let $\lambda \ge K$, $V_1 = \lambda I$, $V_{t+1} = V_t + \sum_{i \in A_t} x_i x_i^\top$ with $|A_t| \le K$ and $\|x_i\| \le 1$. Then $\sum_{t \le T} \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \le 2d \log(1 + KT/(d\lambda))$.*

Proof. Let $B_t = V_t^{-1/2} (\sum_{i \in A_t} x_i x_i^\top) V_t^{-1/2}$, positive semidefinite with $\operatorname{tr} B_t = \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \le K/\lambda \le 1$. So its eigenvalues b_j lie in $[0,1]$, and $\log\det(I + B_t) = \sum_j \log(1 + b_j) \ge \sum_j b_j \log 2 \ge \frac{1}{2} \operatorname{tr} B_t$. Since $\det V_{t+1} = \det V_t \det(I + B_t)$, summing gives $\sum_t \operatorname{tr} B_t \le 2 \log(\det V_{T+1} / \det V_1)$. Finally, $\operatorname{tr} V_{T+1} \le d\lambda + KT$ and AM–GM give $\det V_{T+1} \le (\lambda + KT/d)^d$. □

Proof of Theorem 2. Good event. Order items within a round arbitrarily, and take the filtration that reveals a round's Z values before its rewards. Then $X_{t,m} = Z_{t,m} x_{t,m}$ is predictable, $Z_{t,m} \xi_{t,m}$ is conditionally R-sub-Gaussian, and items with $Z = 0$ contribute nothing. The self-normalized bound [1, Theorem 2], applied to $\theta^\star - \theta_0$ with targets $y - x^\top \theta_0$, gives with probability at least $1 - \delta/4$ for all t: $\|\hat\theta_t - \theta^\star\|_{V_t} \le \beta_T$. The sequence has at most KT terms. So $|x^\top \hat\theta_t - x^\top \theta^\star| \le \beta_T \|x\|_{V_t^{-1}}$ for every x by Cauchy–Schwarz. Combined with the validity event (probability at least $1 - \delta/2$), and writing $\bar\rho_{t,m} = \min(1, \hat\rho_{t,m} + \varepsilon_{t,m})$ and $\bar\mu_{t,m}$ for the clipped upper confidence bound,

$$\rho_{t,m} \le \bar\rho_{t,m} \le \rho_{t,m} + 2\varepsilon_{t,m}, \qquad \mu_{t,m} \le \bar\mu_{t,m} \le \mu_{t,m} + b_{t,m}, \quad b_{t,m} := \min(1, 2\beta_T \|x_{t,m}\|_{V_t^{-1}}).$$

Clipping preserves the lower bound as $\mu_{t,m} \in [0,1]$. In particular $w_t^\star \le \bar w_t$ coordinate-wise.

Per-round regret. Let $S_t^\star$ be optimal for $w_t^\star$. By coordinate monotonicity, the oracle property, and the Lipschitz property,

$$\alpha F_t(S_t^\star; w_t^\star) \le \alpha F_t(S_t^\star; \bar w_t) \le F_t(S_t; \bar w_t) \le F_t(S_t; w_t^\star) + L \sum_{m \in S_t} (\bar w_{t,m} - w^\star_{t,m}).$$

Moreover, $\bar w - w^\star = \bar\rho\bar\mu - \rho\mu = \rho(\bar\mu - \mu) + (\bar\rho - \rho)\bar\mu \le \rho\, b + 2\varepsilon$.

Summation. $b_{t,m}$ is determined by the history before round t, and $\mathbb{E}[Z_{t,m} \mid \text{history}] = \rho_{t,m}$, while Z values within a round may be dependent. So $D_t = \sum_{m \in S_t} (\rho_{t,m} - Z_{t,m}) b_{t,m}$ is a martingale difference sequence with $|D_t| \le K$, and Azuma–Hoeffding gives, with probability at least $1 - \delta/4$, $\sum_t \sum_{m \in S_t} \rho_{t,m} b_{t,m} \le \sum_t \sum_{m \in S_t} Z_{t,m} b_{t,m} + K\sqrt{2T \log(4/\delta)}$. The items with $Z = 1$ are exactly the ones entering V. By Cauchy–Schwarz and Lemma 2,

$$\sum_t \sum_{m \in S_t} Z_{t,m} b_{t,m} \le 2\beta_T \sqrt{KT \sum_t \sum_{m : Z = 1} \|x_{t,m}\|^2_{V_t^{-1}}} \le 2\beta_T \sqrt{2dKT \log(1 + KT/(d\lambda))}.$$

A union bound over the three events finishes the proof, and $\sum_t \sum_{m \in S_t} 2\varepsilon_{t,m}$ is the remaining term. □

Proof of Proposition 8. Write $\lambda = \lambda_k$. The change count $N(s)$ has intensity $\lambda Y(s)$, where Y is the predictable number of items at risk, and $E(s) = \int_0^s Y$. For any fixed real θ, $M_\theta(s) = \exp(\theta N(s) - \lambda(e^\theta - 1) E(s))$ is a non-negative local martingale with $M_\theta(0) = 1$, hence a supermartingale, so $\Pr[M_\theta \ge 2/\delta] \le \delta/2$ at the stopping time by Markov's inequality and optional stopping.

Let $\ell = \log(2/\delta)$ and $\theta = \sqrt{\ell/(\lambda E_0)} = \eta/2 \le \frac{1}{4}$. Taking $+\theta$ and using $(e^\theta - 1)/\theta \le 1 + \theta$ on $[0,1]$: $N \le \lambda E(1 + \theta) + \ell/\theta$. Taking $-\theta$ and using $(1 - e^{-\theta})/\theta \ge 1 - \theta$: $N \ge \lambda E(1 - \theta) - \ell/\theta$. Dividing by λE and using $E \ge E_0$, with $\ell/(\theta \lambda E_0) = \theta$, gives $|\hat\lambda/\lambda - 1| \le 2\theta = \eta$.

Put $s = \hat\lambda/\lambda \in [\frac{1}{2}, \frac{3}{2}]$. By the mean value theorem, for some σ between 1 and s, so $\sigma \ge \frac{1}{2}$, we have $|e^{-s\lambda a} - e^{-\lambda a}| = |s - 1|\, \lambda a\, e^{-\sigma \lambda a} \le \eta/(e\sigma) \le 2\eta/e$, using $\sup_{u \ge 0} u e^{-\sigma u} = 1/(e\sigma)$, for all a at once. □

Proof of Proposition 9. Let each round have m items with equal features, $\mu = 1$, $c = 1$, $B = 1$ and $F(S; w) = \sum_{i \in S} w_i$, with the index of the valid item uniform and independent across rounds. The information a blind learner gets from past rounds is independent of the current valid index, so whichever item it picks is valid with probability $1/m$ and $\mathbb{E} F_t(S_t) = 1/m$, while $\mathrm{OPT}_t = 1$. □

Proof of Proposition 10. $\int_0^\infty e^{-\lambda a} \frac{\beta^\alpha \lambda^{\alpha-1} e^{-\beta\lambda}}{\Gamma(\alpha)}\, d\lambda = (\beta/(\beta + a))^\alpha$. The logarithm is $-\alpha \log(1 + a/\beta) = -\alpha \log a + \alpha \log \beta + o(1)$. A single access at age a has ACT-R activation $B = \log a^{-d} = -d \log a$. □

Proof of Theorem 3. Φ is monotone submodular (facility location), $\Phi(\emptyset) = 0$ and $\Phi(X) = 1$. For the greedy sequence with density-maximal g_i, as in the proof of Theorem 1, $\Phi(Z^\star) - \Phi(Z_{i-1}) \le R^\star \Delta_i / c(g_i)$, so $d_i := \Phi(Z^\star) - \Phi(Z_i) \le (1 - c(g_i)/R^\star) d_{i-1} \le e^{-c(g_i)/R^\star} d_{i-1}$ whenever $d_{i-1} > 0$, using $1 - x \le e^{-x}$ for all real x. If $d_{i-1} \le 0$ for some i, then $\Phi(Z_\ell) \ge \Phi(Z^\star)$ by monotonicity. In every case $\Phi(Z_\ell) \ge (1 - e^{-R_\ell / R^\star}) \Phi(Z^\star)$. Substituting $\Phi = 1 - D$ gives the first claim.

For the second, unit vectors satisfy $\|e_x - e_z\|^2 = 2(1 - \langle e_x, e_z \rangle)$. So $|\langle e_q, e_x - e_z \rangle| \le \sqrt{2(1 - s(x, z))}$, which bounds the score change by $\ell_{\mathrm{u}} \sqrt{2(1 - s(x, z(x)))}$. Average over p and apply Jensen's inequality to the square root, noting that $\mathbb{E}_p[1 - s(x, z(x))] = D(Z_\ell)$. □

Proof of Theorem 4. Condition on r and let T be the time until the first gap of length τ in a Poisson process of rate r started at the arrival. Let $A \sim \mathrm{Exp}(r)$ be the first access. If $A > \tau$ then $T = \tau$, otherwise $T = A + T'$ with T' an independent copy of T by the memoryless property. So $\mathbb{E}T = \mathbb{E}\min(A, \tau) + (1 - e^{-r\tau})\mathbb{E}T$. With $\mathbb{E}\min(A, \tau) = (1 - e^{-r\tau})/r$, this gives $\mathbb{E}T = (e^{r\tau} - 1)/r$.

The pairs (arrival time, lifetime) form a marked Poisson process with i.i.d. marks. By the marking theorem, the number of items alive at t is Poisson with mean $a \int_0^t \Pr[L > u]\, du$, which converges to $a\,\mathbb{E}L$ under the finiteness assumption. □

Proof of Proposition 7. For any set S, $|\sum_{m \in S} p_m - \sum_{m \in S} \hat{p}_m| \le e(S)$. Hence $\sum_{\hat{S}} p \ge \sum_{\hat{S}} \hat{p} - e(\hat{S}) \ge \alpha \sum_{S^\star} \hat{p} - e(\hat{S}) \ge \alpha \sum_{S^\star} p - \alpha e(S^\star) - e(\hat{S})$, where the middle step uses that $S^\star$ is feasible. For $\alpha = 1$, the items in $S^\star \cap \hat{S}$ cancel: apply the same chain to $S^\star \setminus \hat{S}$ and $\hat{S} \setminus S^\star$, using $\sum_{\hat{S} \setminus S^\star} \hat{p} \ge \sum_{S^\star \setminus \hat{S}} \hat{p}$, which holds because $\hat{S}$ is optimal for $\hat{p}$.

For $\alpha = \frac{1}{2}$ with an additive objective: let density greedy stop at the first item g that does not fit, with accepted set G, ignoring later items, which only add value. The items are in density order, so the value of $G \cup \{g\}$ is at least the optimum of the fractional relaxation, and so at least the integer optimum. Thus $\hat{p}(G) + \hat{p}(g) \ge \mathrm{OPT}$, and the better of G and the best singleton has value at least $\frac{1}{2}\mathrm{OPT}$. □